This is an excerpt from Frank Elavsky's dissertation on *Tool-making as an Intervention on the Accessibility of Interactive Data Experiences*, which can be accessed in full at this archival link (once published):

http://reports-archive.adm.cs.cmu.edu/anon/hcii/CMU-HCII-26-103.pdf

This document contains the following sections:

- Abstract
- Table of contents
- Chapter 1: What do we mean by "accessibility" and "tools?"
- Chapter 2: Background & Related Work
- Chapter 3: Overview of Contributions
- Chapter 5: *Data Navigator*: Low-level Tooling for Creating Navigable Data Structures
- Chapter 9: Findings (Sections 9.2 and 9.3 only)
- Chapter 10: Discussion & Future Work (Sections 10.1 and 10.3 only)
- References

This document does not contain the following chapters:

- Chapter 4: *Chartability*: Heuristics as a Tool and Resource
- Chapter 6: *Skeleton*: Visual Authoring of Non-visual Data Experiences
- Chapter 7: *Cross-perception*: Rethinking Input Design Towards Blind Analytical Interaction
- Chapter 8: *Softerware*: Enabling Personalization of Interactive Data Representations for Users with Disabilities
- Chapter 12: The *Softerware* Option Taxonomy
- Chapter 11: Biographical Sketch

Contents

List of Figures

Abstract

In this dissertation, we contribute practical advancements in tool-making as an intervention on the accessibility of interactive data experiences. The thesis of this dissertation is as follows:

This dissertation argues that the tools practitioners use to build interactive data experiences are themselves sites where accessibility barriers are produced, prevented, or alleviated for both end users and authors. This work contributes five tools, Chartability, Data Navigator, Skeleton, Cross-perception, and Softerware, that collectively center accessibility work on the empowerment of disabled and non-disabled practitioners across the full arc of evaluation, data navigation, analytical interaction, and personalization.

Rather than framing accessibility research solely around ideal experiences for end users with disabilities, this thesis investigates why accessibility work is so difficult for the practitioners who build interactive data experiences and what tool-making can reveal about those difficulties. We organize this investigation around four domains where practitioners face the most persistent challenges: evaluation, navigation, interaction, and personalization. *Chartability*, a heuristic framework, contributed first and mapped the full landscape of accessibility barriers. *Chartability* helped us to identify and prioritize our three latter domains (navigation, interaction, and personalization) as the areas where the most severe gaps remain in practitioner work. *Data Navigator* and *Skeleton* then investigate navigation, finding that practitioners struggle because navigation structure has no visible, manipulable representation in their workflows. *Cross-perception* engages interaction, demonstrating that blind data analysis has been constrained by existing tools and that a new interaction design framework can reshape what analytical work is possible. *Softerware* addresses personalization, revealing that access needs genuinely conflict across users and that meaningful personalization requires system-level infrastructure that does not yet exist in practitioner tooling ecosystems.

Combined, these contributions provide empirical insights and practical advancements in the state of the art for tooling that bridges gaps in current accessibility practices in visualization and data science. Our work ultimately enables people with and without disabilities to better evaluate barriers in, analyze with, design for, develop, and personalize interactive data experiences. We demonstrate that tool-making is a productive intervention that both engages accessibility barriers and elucidates why those gaps exist in practitioner work.

Part I

Introduction

Chapter 1
What do we mean by "accessibility" and "tools?"

This thesis is a body of research situated within the existing research area focused on making interactive data representations more accessible for people with disabilities. Much of the work in this existing area is situated within the context of making interactive data *visualizations* accessible, particularly (but not exclusively) for people who are blind. Our work, contributed here in this thesis, is focused on using *tools* as a specific intervention and sub-area of study for making interactive data *representations* accessible for people with disabilities, broadly speaking. ("Representations" here is an intentionally broader term than "visualizations," which are exclusively visual representations of data.)

Before we begin, two things must be understood up front, or else the rest of this thesis could be interpreted with disruptive assumptions: we must interrogate the phrase "making visualizations accessible" and unpack why *tools* are a meaningful area of study.

1.1 Is "accessible visualization" really an oxymoron?

The first assumption that must be disrupted is perhaps the motivating cornerstone of this research, which is that the phrase "making visualizations accessible," while a noble goal, is not the semantically correct phrasing nor precisely what describes our work. This can be misleading. We do use the phrase "accessible visualization" but will admit that this seems to confuse certain people with very particular opinions about things. We will clear this up.

Villains of our field's past have written incendiary and ableist perspectives on why "no forms of data visualization, not just dashboards jam-packed with graphics, can be made fully accessible to someone who is blind," and that "[a blind man] will never be able to analyze data as I do visually, because many aspects of vision cannot be duplicated by his other senses" [34]. However, this position misunderstands what the goal of accessibility is, and arguably even what the goal of visualization itself is.

Making visualizations accessible *isn't* about the visualization, it's about making the outcomes of the visualization accessible.

Visualizations are ubiquitous and paramount for decision-making. However, the *artifact* that is a visualization is not even the goal of the act of visualizing: developing understanding, insight, confidence, and communication among and between human beings are the goals of visualization. Visualization is about making data easier to use for all kinds of things. Yes, our visual system enables us more than any other form of sensory cognition that we have [18, 41, 126]. But we aren't trying to make sight itself accessible. We are trying to make it possible for people to make meaningful decisions, gain valuable information, build conjectures, and effectively communicate with others.

Many, many people who I, Frank, have spoken to over the course of my career, even before embarking on this thesis journey, misunderstand this simple fact: making a "visualization" accessible *isn't* about the visualization itself but rather making what the visualization is meant to

accomplish accessible. It's about equal outcomes, not equal interactions with an artifact.

People with disabilities are no small portion of the world's population. In the United States, 26% of people self-report living with at least one disability that affects their daily lives [100] and all of us will eventually age into disability (if we are lucky to live a long life).

People with disabilities deserve to participate fully in life. They deserve financial independence. They also deserve loving care and interdependence [8, 65, 97]. People with disabilities have a right to make informed decisions, to know about the status of a global pandemic, and to have an understanding of local and national politics [33]. While we use visualizations to navigate all of these domains, the goal is not to make the charts and graphs themselves somehow equally useful to all people. That would be a false measurement of success.

Our goal then, is measured by the success of lives led by people with disabilities [131]. Many other measurements are just metrics along the journey towards that goal. We then ask: Can people with disabilities also use data to live full lives? Can they make *fast* decisions based on data? Meaningful, careful, *slow* decisions? Communicate complex ideas? Crunch and clean data, develop models, find errors, and build hypotheses? Can they have memorable, immersive, beautiful, aesthetic experiences with data too [65]? Ultimately, our aim is not accessible "visualizations" but equitable and accessible *outcomes*, everything an interactive data experience *accomplishes* for the people who use it.

Again, if the goal of accessible visualization were about visualizations themselves, then the correct course of action would be one framed by the medical model of disability [91]: that there is a normative state of behavior and capability (in this case, it would be "normal" to be able to read a visualization and make a decision) and any deviation from that norm must be corrected. This framing first assumes that the visualization should not be altered or improved. And then this framing puts the burden on the bodies of people with disabilities: that they must be "fixed" and given sight or brought to some equivalent state as someone who is "healthy," normal, and sighted. Plenty of scholars have already discussed why this framing is a problem, not only because it places undue burden on people with disabilities and produces pathologies and hierarchies of disability, but also because it is fundamentally not economically or ethically feasible.

So we then turn to other models of disability, such as the social model. The social model is heavily discussed by disability scholars and is not the end-game or last and total way of thinking about disability [91, 102, 104, 124, 154]. But the core motivation is that society, not medicine, is also a path towards solving problems that people with disabilities face. A few important concepts and concretely actionable things come from the social model that can help motivate the work of this thesis.

First, we look to the historical birth of the social model of disability: in the 504 sit-ins that took place in the United States in 1977. Cities had curbs and curbs are a barrier for people who use wheelchairs. So protests happened because decisions were being made without people with disabilities at the table. In this instance, people acknowledged that political power was an exclusive club and fought to ensure their cry "nothing about us, without us!" materialized.

And this leads us to the first and most-foundational philosophical framing for this thesis: that our *artifacts*, these things we've created from curbs to data visualizations, can become *barriers* for people with disabilities. And it is then the artifact, not the body of the person with a disability, where disability is produced in this model. Rather than a comparison to a normative state as a

way to frame disability (the medical model), we instead must observe and evaluate material outcomes based on human-made problems.

So, the social model is framed around society "solving" inequities: we get involved and make political and legal change tangible. But a second model also emerges from within the social model: one where we can now frame *who is first responsible* for repair: the curb designers and implementers.

And knowing who is first responsible for access leads us into the moral and ethical imperative that motivates this thesis: the builders and makers of visualizations are ultimately the ones who provide exclusive value for only a subset of people: those *without* disabilities. **We must first change how builders and makers do their work.**

So the phrase "accessible visualization" is really about recognizing that visualizations produce barriers for people. That means that it is our ethical imperative, as builders and makers, to fix them. And that act of fixing barriers leads us away from mere visual representations of data into a wide variety of other senses and interaction modalities. There are many paths forward towards fuller and more-equitable lives led by people with disabilities.

1.2 On *tools*, *tool-making*, and *human-tool* interaction

Then the act of making becomes immensely important: we, the builders and makers of our world, need to get things right; there is a risk involved when making things that we will exclude people with disabilities. We need to make sure that we build a better world than the one we have now. We must care for new things we create and tend to the repair and maintenance of what we've already made. And this ethical imperative leads us to the topic of *tools* and *tool-making*.

So the second thing that must be understood before we embark on this thesis is that *tools* are not the same as *solutions* or *applications*. Sometimes tools can be used to *solve* things and are certainly, in ideal circumstances, *applied* in various contexts. But understanding the role of the "tool" in human-tool interaction is paramount for engaging in the work of making anything accessible for people with disabilities.

We use tools to shape our world, break old things, and make new things. But a tool, like the hammer (as an example), does not inherently *solve* something like homelessness. But a hammer can be used to build homes if there are social policies in place and proper resources allocated. This means that for the success of tooling, there is often a larger material, social, legal, and policy reality that supports and necessitates those tools. This thesis will not be focusing on changing the upstream dependencies, but optimistically operating as if they were true (or will be true in time).

However, in some cases, tools can *destroy*. The hammer has a claw and can easily pry apart boards and tear down homes. So tools carry potential to do all kinds of things, both good and bad, and how a tool is used is often open-ended, variable, and heavily dependent on socio-technical realities. Tools participate in personal and political agendas [149] and are sometimes, for this reason, regulated or made proprietary and controlled by powerful entities [44, 146].

So tools are not without any sort of ethics. We cannot just blame tool-users for outcomes when much of a tool depends on these larger systems and structures. Technologies (tools included) encode the assumptions and biases of their *creators* as much as, if not more than, their

users. Tools that build things for others to use can be loaded with assumptions about what people are *able* to do [150] and also rules and guardrails about what anyone downstream from that tool's design *should* do [44, 145]. These assumptions, biases, and rules *limit*, *enforce*, *magnify*, *exclude*, and *enable* what a tool-user is capable of.

Tools for visualizing data are a perfect case study in this problem: virtually every major data visualization library, application, or software ever made was made entirely with the assumption that data should be transformed into visual representations. This is a reasonable assumption, since virtually all of the tool-makers are sighted and visualization is incredibly helpful to our cognition of and communication with data [37].

So data visualization, as a field, has focused its tool-making efforts on reducing the difficulty involved in visualizing data. Some visualization tools are concise [111], others are lower level but much more expressive [14]. Tool-making in visualization has focused on making it easier to scaffold a wide variety of interactions both with the visualizations as well as with their underlying data models [54].

However, as time has moved on, people began to speak out about color-vision deficiency in data visualization. Some people, primarily those with X/Y chromosomes (largely men) who are of European ancestry, have a deficiency in their ability to perceive certain colors. Then a plethora of research arose that began to look into the barriers that folks who are colorblind face in data visualization. As a result, our practices and tools improved. We began to educate practitioners, develop new color palettes, researched new methods for testing our designs, and built new systems for handling automatic color encoding. Our tools evolved.

But now data visualizations have arguably become ubiquitous in daily life. By comparison, we have far more tools now for making visualizations quickly and easily than we do for representing data in non-visual ways. We also have far more research, relatively speaking, into how sighted end users interact with visualizations.

So this thesis engages gaps that arise in this space: Practitioners face immense challenges when crafting accessible data experiences. We first need to educate practitioners on what accessibility barriers actually are in interactive visualizations. Then, we must help them engage the hardest barriers in this work and create building blocks that help them to construct navigable data experiences, build design frameworks that can inform entirely new kinds of data interaction, and develop software systems for end-user personalization and agency. Our research seeks to advance the state of the art in tools that assist in accessible data interaction while also using tool-making as an intervention that helps us to better understand and characterize *why* and *how* data practitioners face barriers themselves in this work.

Chapter 2
Background & Related Work

This chapter sets up the story that the rest of this thesis engages. It makes three moves. First, we ground the idea that tools are not neutral instruments: tools shape the work and the workers who use them, which is why tool-making is a meaningful site for accessibility intervention. Second, we survey the long conversation between accessibility and data visualization and its history across modalities, including engaging critical voices that intersect with visualization and accessibility work which inform, support, and hold in tension the work of this thesis. Lastly, we argue that the hardest remaining challenges cluster in three domains, navigation, interaction, and personalization, and we signpost which chapters engage each of these.

2.1 Tools and the People Who Use Them

2.1.1 Studying Practitioners and Their Work

Human-computer interaction has a long tradition of studying the people who make things. Ethnographic and case-study methods immerse researchers in maker, DIY, and assistive technology communities to understand how practitioners actually work, especially when the work is new and unfamiliar to them [64, 66, 67]. Participatory design and co-creation bring practitioners and end users directly into the research process, surfacing how designers shift their thinking when they encounter novel problems [48, 108, 125]. And a growing body of design-based research aims not merely to fill gaps in existing practice but to extend what practitioners are capable of: scaffolding experimentation, supporting learning, and amplifying creative and technical expression [22, 46, 69, 74, 133].

This thesis sits inside that tradition but takes a particular methodological stance: we study practitioners *through their tools*. Each project in this thesis builds a tool, places it in front of working practitioners, and observes what the tool reveals about why accessibility work is hard for them. Sometimes the tool is a resource practitioners apply in their real environments, sometimes a system they build with, and sometimes a probe whose main purpose is to make an invisible part of their work observable. Tool-making here is both an intervention and an instrument of inquiry. That stance only makes sense, however, if tools really do shape work in a deep way. So before surveying the accessibility and visualization literature, we want to engage that claim seriously.

2.1.2 How Tools Use Us

We tend to talk about using tools. We talk much less about how tools use us. A chair is a tool for sitting, but a lifetime of chairs trains a body into the chair's posture. A pickaxe is a tool for extracting ore, but the mine organizes the miner: their hours, their movements, their risks, and what counts as a day's work. Tools come with a direction already built in, and the people who take them up inherit that direction. As a participant in Hohman et al's work on model compression in practice remarked, "Better tools make it easy to do correct things and harder

to do incorrect things“ [63]. Tools are made precisely with the intention to shape us and our behavior.

Sara Ahmed gives this intuition theoretical teeth. In her phenomenology of orientation, objects are not passive things waiting to be used: they direct bodies, making some actions feel near and natural while other actions fall out of reach, and the social world is full of what she calls straightening devices, arrangements that keep bodies aligned with the paths already laid down for them [1]. In her later work she traces how *use* itself is formative: things become shaped by their use, “fit” for some users and not others, and a history of use leaves a trace that directs how a thing will be used next [2]. Bowker and Star make a parallel argument at the scale of infrastructure: the classification systems and standards embedded in our information systems are largely invisible scaffolding, and every standard valorizes some point of view while silencing another [15]. Assumptions about ability are among the assumptions most commonly baked in: tools encode what their makers believe people are able to do [150].

Visualization research has begun to examine its own tools in these terms. McNutt’s survey of 57 JSON-style domain-specific languages for visualization shows that each grammar embodies contested decisions about who its user is and what that user may express, with recurring tensions between formal and colloquial specification and between competing user types [94]. His dissertation develops the larger argument: the design of visualization languages, and of the tools that surround them, directly shapes and can either enhance or constrain end-user agency, motivating systems that offer assistance while respecting the user’s intent [95]. In other words, the field’s own instruments are straightening devices. Every charting grammar that assumes a sighted reader, every renderer that emits unstructured pixels, quietly directs practitioners away from accessible outcomes, not by forbidding them but by making them feel out of reach.

Tool users are not powerless in this relationship. A rich literature on appropriation documents how people adapt tools beyond their designers’ intent, and how that emergent use can be a source of design knowledge in its own right [26, 29, 107, 130], with some work building general theory from the study of emergent and generative tool use [7, 10]. HCI’s systems community has likewise developed methods for building and studying tools deliberately: piggybacking on existing systems to study an improvement in context [45, 57], and contributing toolkits whose value is argued through demonstration, usage, technical performance, or heuristics. Ledo et al., analyzing 68 toolkit papers, found these four evaluation strategies dominate, with demonstration by far the most common [80]. That finding will matter later in this chapter: a field can produce many demonstrated toolkits while producing very little maintained infrastructure.

The takeaway from this section is the premise the whole thesis rests on: tools direct work. If the tools of data visualization currently straighten practitioners toward inaccessible outcomes, then re-orienting those tools is not a cosmetic fix. It is an intervention at the place where practice is formed. The rest of this chapter examines what the field has learned about accessible data experiences, and why that learning has so far failed to re-orient the tools.

2.2 The Conversation Between Accessibility and Data Visualization

Accessibility and data representation have been in conversation far longer than the modern visualization community has existed, and that conversation has two important threads. The first is a history of techniques: a steady accumulation of ways to carry data to people through senses and devices other than a sighted eye on a screen. The second is a critical lineage, largely led by disabled scholars and disability studies, that questions how that technical work is framed, who leads it, and who it serves. This thesis tries to take both seriously, so we survey them in turn.

2.2.1 A Short History Across Modalities

Tactile representation is the oldest thread. Tactile maps and graphics for blind readers date to at least the 1830s [47], and tactile sensory substitution for charts has been studied in the computational sciences since the early 1980s [113]. Practice in this modality is unusually mature: tactile and braille standards are robust and well-maintained [6], although their assumptions (embossed paper, fixed layouts) transfer only partially to digital, interactive contexts. Research has worked on that seam, for example by augmenting tactile graphics with audio annotations [5], and industry accessibility teams have published candid experience reports on what it takes to produce tactile graphics inside a real design workflow [24]. Recent work continues to refine the perceptual foundations of non-visual structure [96].

Sonification has a similarly long arc, and because it is a substantial and active research community in its own right, it deserves explicit treatment here. Mansur et al.'s sound graphs demonstrated in 1985 that line data mapped to pitch over time could communicate trends to blind listeners [90]. Subsequent decades built out the empirical base: cross-modal comparisons of auditory and visual graphs [36], non-visual presentation of structured information [17], techniques for scanning and comparing multiple data series by ear [93], and interactive sonification of geo-referenced data [155]. The community consolidated its methods and theory in the Sonification Handbook [56]; Walker and Nees's theory chapter formalizes sonification as a mapping problem from data dimensions to auditory dimensions, emphasizes context cues that play the role axis ticks play in vision, and stresses that interaction, not just playback, shapes what listeners can extract [141]. Sonification research has also been a site of methodological progress, including co-design with blind and low vision collaborators [23].

What is most relevant for this thesis is the recent trajectory: sonification is increasingly integrated into multimodal, interactive systems rather than offered as a standalone substitution. VoxLens adds summaries, sonification, and voice queries to web visualizations through a JavaScript plug-in, and in a study with 21 screen reader users improved information-extraction accuracy by 122% and interaction time by 36% over conventional interaction [118]. Chart Reader, co-designed with screen reader users, combines structured navigation with sonification and author-configurable data insights [132]. Auditory maps have reached usable maturity for wayfinding and choropleth reading [12]. Notably, sonification is also one of the few accessible-visualization techniques to cross into production: Apple shipped Audio Graphs in 2021 as a VoiceOver feature with a public developer API [4]. And Umwelt re-frames the relationship en-

tirely, treating sonification, textual structure, and visualization as coequal representations derived from a shared abstract model in an accessible authoring environment [158]. Sonification's trajectory, from output substitution toward interactive structure and authoring, previews the argument this chapter is building.

Textual description is the most widely deployed modality, and research has moved it well past "write some alt text." Lundgard and Satyanarayan's model of semantic content distinguishes the levels of meaning a description can carry, and shows that blind and sighted readers value different levels [84]. Jung et al. offer evidence-based guidance on description length, plain language, and the ordering of information [72]. Kim et al. collected the questions screen reader users actually ask of visualizations, pointing toward more dialogic, query-driven access [76], and Chundury et al. deepen our understanding of how sensory substitution choices should be grounded in users' practices [21].

Navigation and screen reader interaction is the youngest thread and the one closest to this thesis. Foundational work made statistical and mathematical content traversable by screen reader [42, 123]. Studies of screen reader users' experiences with two-dimensional digital content, including visualizations, documented both the depth of exclusion and the strategies users develop anyway [112, 117]. Zong et al. then gave the area its clearest design vocabulary: structure (how a visualization's entities are organized for traversal), navigation (the operations that move through that structure), and description (what is spoken at each position), showing through co-design that screen reader users want rich, structured exploration rather than a single summary [157]. Olli operationalized part of this insight as an open-source adapter that renders structured traversal for existing charting libraries [13]. A small number of production systems, notably Highcharts, SAS Graphics Accelerator, and Visa Chart Components, have shipped meaningful accessibility functionality [58, 110, 135]; they remain exceptions, and Chapter 4 examines that practitioner tooling landscape in detail.

Across all four modalities the same pattern recurs: each began with converting visual output into another channel, and each has progressively rediscovered that access is not a conversion problem but an interaction problem, turning toward structure, navigation, interactivity, and user control. The research itself has shifted accordingly: where early decades produced perception studies and one-off substitution systems, the last several years have produced co-designed interactive systems, design dimensions, and authoring environments. That convergence is part of why we argue, later in this chapter, that the remaining hard problems are not modality problems at all.

2.2.2 The Split Between Research and Practice

Before turning to the critical literature, it is worth naming that research and practice participate in accessibility at very different speeds, through very different methods, and with very different artifacts.

Accessibility *practice* organizes itself around standards. The Web Content Accessibility Guidelines carry legal and policy weight across much of the world [136], and a profession of accessibility specialists exists largely to translate such guidelines into concrete implementations. One example of this work would be ensuring that interactive elements carry functional semantics that assistive technologies can interpret, which is a foundational, simple, and necessary requirement for accessibility [138].

Standards are powerful precisely because they are consensus artifacts, or *politically mediated documents*, and for the same reason they are inherently reactive: developing, vetting, and formalizing them takes years, so they chronically trail the technologies they govern and are subject to intervention by individuals and organizations. Interactive data visualization sits at a painful point in that lag. Charts are semantically dense, interaction-heavy, and rendered through technologies the guidelines were not written for, so a practitioner who consults the standards in good faith finds little that speaks to their actual artifact. Robust evaluation in practice therefore leans on expert judgment and, in the best cases, direct involvement of people with disabilities, both of which are scarce resources [101].

Technical accessibility and assistive technology *research* meanwhile (outside of the previously mentioned work on modalities and our empirical record), moves quickly and publishes minimally-viable prototypes. These prototypes are an artifact practice largely cannot consume because it requires technical translation and methodological generalization from the small scope of an artifact into a system-wide implementation [35]. A practitioner cannot install a paper; a paper must be read and the artifact carefully dissected before being reconstructed or replicated. The result is a conversation in which researchers accumulate evidence and demonstrations on one side, practitioners wait on standards that lag by a decade on the other, and very little crosses between them. This split is not unique to visualization, but visualization makes it unusually visible, and closing it from the tooling side is the project of this thesis.

2.2.3 Empirical Depth, but a Lack of Applied Work in Production

What exists, first, is empirical breadth. Surveys and meta-analyses have mapped the research space and its biases: accessibility research broadly concentrates on blindness and on computer output [87], and visualization accessibility research mirrors that concentration, with vision-related work dominating while motor, cognitive, neurological, and vestibular access remain nearly unstudied [77, 92, 148]. Even within the well-studied territory the breadth has a shape: blind and low vision people are routinely researched as one population despite using different assistive technologies and different interaction practices [129], and the prototypes the field celebrates are overwhelmingly built for screen reader interaction in particular.

Second, the field has produced exemplary prototypes. The systems surveyed above, such as VoxLens, Chart Reader, Olli, and Umwelt, are rigorous, validated, and often co-designed with disabled collaborators.

What barely exists is the other half of the pipeline: practitioner tools in production environments, and practitioner action at scale. Studies of deployed visualizations and of the people who build them find that accessible outcomes remain rare and that practitioners struggle even when motivated, gathering fragmented guidance themselves with little way to judge its quality [71, 119]. Mainstream visualization tooling has invested its enormous momentum elsewhere, in expressiveness, scale, and interactive speed [14, 54, 111], with accessibility absent from core abstractions and left to individual developers, libraries, or projects. Research prototypes, meanwhile, rarely become infrastructure: toolkit research is evaluated primarily by demonstration [80], and most demonstrated systems are archived with their papers.

This asymmetry is the load-bearing observation of this chapter: **the field has a breadth of empirical work, yet it lacks practitioner tools in production environments and practitioner**

action. A practitioner today inherits strong evidence about what good access looks like but material change remains far behind. The natural next question is to investigate why this is the case, and whether and how tool-making can play a role in practitioner action.

2.3 Critical Voices and Work Along the Periphery

The second register of the conversation is critical, and although it is sometimes treated as peripheral to systems research, it sets the terms under which systems research should be read. We treat it here as load-bearing. This section will be brief, but essential.

The disability rights movement crystallized its central demand in the principle "nothing about us without us:" people with disabilities must hold power in the decisions and designs that concern them [20]. HCI and accessibility research have been working through what that demand means for our field's methods, authorship, and self-conception [62, 124, 127, 128], including calls for a crip HCI and crip visualization that engages disability on disabled people's own terms [65, 147] and concrete practices such as crediting disabled participant-contributors by name [156].

Several concepts from this literature directly shape this thesis. Mingus names *access intimacy*, the felt sense of someone genuinely getting your access needs, to insist that access is relational rather than merely technical and logistical [97]; Bennett et al. build on that disability justice lineage to propose *interdependence* as a frame for assistive technology research, countering the field's fixation on independence [8]. Hamraie and Fritsch's crip technoscience manifesto insists that disabled people are experts and designers of everyday life, and re-frames access not as a finished state but as friction, something contested and negotiated [52]. Hamraie's history of universal design develops *access-knowledge*, showing that what counts as access has always been produced through political struggle rather than neutral technical progress [49, 50]. Bennett et al.'s biographical prototypes push against savior narratives in design by recognizing the prototyping disabled people already do in their own lives [9]. Shew names the broader cultural failure *technoableism*: the belief that technology will "fix" disabled people, held mostly by people who never asked disabled people what they want [120]. And within visualization specifically, Hsueh et al. bring crip technoscience to bear on data experiences, articulating how *access friction* arises when one design that includes some people excludes others [65].

We read this literature first as a set of constraints on what this thesis may claim. It is why we reject techno-solutionist framing outright: no tool in this thesis "solves" accessibility, because access is negotiated among people, institutions, and policy, and tools do not meaningfully redistribute power entirely on their own. It is also why access friction appears throughout this thesis as a design reality rather than an edge case, and why personalization is treated in Chapter 8 as an ethical question and not merely an intellectual problem to solve. And this body of work imposes a boundary we want to name early: a thesis about practitioner tooling mostly serves practitioners building *for* disabled users. The later chapters, Cross-perception and Softerware, begin to shift who holds the power to shape our interfaces. The critical literature is the measure of how far there is still to go.

We also read this literature as relatively divorced from accessibility work in practice. Critical scholarship has even been critical of "access" as a framing and its shortcomings [49, 50, 51, 101, 147]. Yet, practitioners are building systems all over the world that must be made accessible.

And rather than argue that people with disabilities should not be included in the design and development of new projects, instead we argue that, at least in some cases, the empirical record of what people with disabilities want and need is already enough imperative for builders and designers to begin working. For something new and unexplored? Yes, people with disabilities should be included. But what if we have a sufficient breadth and depth of guidance from people with disabilities already? Aren't we then ready to take action and begin practical application? *Chartability* in Chapter 4 takes this stance: people with disabilities have already said enough for us to build out comprehensive guidelines and an evaluation framework. Between standards, research, and community practice, we have an authoritative set of criteria we can test against. This should be our starting point; the bare minimum. And *Data Navigator* in Chapter 5 didn't seek to build a novel interaction but rather build a better system than what currently exists, following existing empirical work that laid the foundation for rich navigation as a necessity [157]. And in Chapter 5 we *do* demonstrate that our system enables novel interaction designs and how practitioners can approach co-design in a simple, effective, and thoughtful way with partners with disabilities. Chapter 6 sought to investigate the barriers that sighted practitioners (without disabilities) face, which is outside of the scope of critical disability literature, and Chapters 7 and 8 then centered on the experiences of blind and low vision people, and how they might have interactive and innovative power over their experiences.

2.4 Why Navigation, Interaction, and Personalization

If the gap were uniform, any contribution would do. It is not uniform. Looking across the empirical record, the prototypes, and the practitioner studies, the places where practitioners most consistently lack the means to act cluster in three domains, and each domain fails for a different kind of reason.

It is worth being explicit about why these three and not others, because the field's other major needs are of a different kind. Description is the best-served domain: practitioners have models, guidance, and concrete heuristics for what to write [72, 84], and what remains is largely a matter of doing it. Evaluation is hard, but it is hard as a knowledge problem, a matter of gathering, judging, and applying scattered guidance, and a well-built resource can carry a practitioner a long way; Chapter 4 contributes exactly such a resource and uses it to map the territory. *Chartability*, from Chapter 4, ultimately ends up framing the rest of this thesis. As we did work alongside our co-designers, clients, and industry partners over the years, applying *Chartability* to their work, it became clear that three major barriers emerged as the most pernicious (or least-resourced) for practitioners to address.

Navigation, interaction, and personalization are different in kind: in each of them, a practitioner who knows precisely what good access requires still cannot build it, because the means do not exist in their tools. Guidance cannot substitute for means. These areas then serve as opportunities to innovate, to build better, more general, or more specific tooling.

We unpack these three tooling gaps more below.

2.4.1 Navigation

The empirical case for structured navigation is now strong. Zong et al. showed through co-design that screen reader users want to traverse a visualization's structure, not only hear a summary: structure, navigation operations, and description together determine the quality of the experience [157]. Systems work has begun to deliver this for specific stacks, with Chart Reader designing navigation experiences with screen reader users [132] and Olli adapting existing charting libraries into traversable structure [13]. What practitioners still lack is twofold. They lack low-level building blocks: a way to give *any* data interface, whatever its renderer or framework, a navigable structure with arbitrary shape, rather than the fixed hierarchies particular systems assume. And they lack any visible, manipulable representation of navigation structure in their workflows: a sighted practitioner can see a color palette and act on it with existing tooling but cannot see a navigation graph and act on it, which makes navigation design incredibly burdensome to iterate on and wholly downstream from other visual work. Chapter 5 (*Data Navigator*) contributes the structural substrate, and Chapter 6 (*Skeleton*) contributes the authoring representation; their chapter-level related work sections cover visualization toolkits and authoring systems more in depth.

2.4.2 Interaction

History shows that input design, not just output conversion, changes what is possible. Kane et al.'s *Slide Rule* contributed multi-touch techniques that let blind users operate touchscreens, outperforming a button-based baseline and prefiguring techniques that later appeared in commercial screen readers [73]. That simple lesson has not yet reached data analysis. Accessible visualization interaction today is overwhelmingly what we call *access-oriented*: it exposes information that is already there, through navigation, sonification, or description. These methods are *interactive* as existing work as argued [157], but we argue that the interaction is focused on perception of the representation and not enabling analytical interaction with and manipulation of the information that lies beneath the representation. Analytical work of the kind sighted analysts perform with cross-filtered, coordinated views depends on rapid, continuous loops of input and perception [54, 82], and a screen reader's serial, query-at-a-time interaction may be structurally unable to sustain such loops. Fast, complex manipulation of a data space is presently made inaccessible by the primary assistive tool used by blind analysts. Rethinking input for blind analytical interaction, beyond an *access-oriented* framing, is nearly unexplored territory. Chapter 7 (*Cross-perception*) takes this on, formalizing a design framework and contributing a hardware prototype; that chapter's related work covers tactile and haptic input devices in more detail.

2.4.3 Personalization

Accessibility research has long argued that systems, not users, should do the adapting. Put simply, this means that systems must be capable of adaptation. Arguably, this is the "soft" part of *software*. Yet, this softness can create conflict in system design, especially when an entire field of work must refactor their dominant tooling in order to enable system adaptation by end users. Visualization faces this exact challenge.

Existing work has demonstrated that interfaces adjusted to fit a person's abilities and preferences measurably improved speed, accuracy, and satisfaction for users with motor impairments compared to default interfaces [38], and ability-based design generalized the stance into principles [150]. Good systems are made to fit a user's abilities and remove barriers that they face. It could be argued, in broad strokes, that the field of HCI centers on the goal of *fit* between a user and a computational system.

Yet, visualization toolkits are largely not made to be personalized by end users. They are rigid and *hard*; lacking that softness required for end-user manipulation. Visualizations are typically built with a *universal design* [50] approach, when being made accessible (if being made accessible at all): one design is intended to fit as many people as possible. In visualization, recent work shows screen reader users benefit from customizing even small things, like the textual tokens spoken during navigation [70]. Yet a significant gap exists that enables visualization systems to adapt to personalization at scale and for many different people. How should these systems be built and what would they require? Which dimensions and features and functionality of data visualizations *should* be manipulable and customizable to fit end users? How should systems be constructed in order to make personalization effective and useful? And ultimately: how do we make visualization toolkits *softer*?

Some existing work on customizable accessibility settings identifies what drives people to adopt or abandon personalization features [152], but not all of this work neatly translates into visualization system design and development. Additionally, and fundamentally, we simply lack an empirical record of what exactly different people with disabilities would even want or need to manipulate. Presently, no mainstream visualization library or system offers practitioners infrastructure for end-user preferences over data representations: no shared option vocabulary, no persistence, no way to reconcile preferences with authorial defaults. Most visualizations become completely static in their design. Those with manipulable elements are only manipulable through hacking or manual overrides, placing a significant burden of access on end users, exactly where ability-based design says it should not [150]. Chapter 8 (*Towards Softerware*) engages this domain in collaboration with a production charting library, considered by the empirical record to be the most-accessible toolkit according to present evaluation methods by a wide margin [78]; Chapter 8's related work section covers adaptive systems and personalization research in more depth.

2.5 What to Take Away

Five things from this chapter frame everything that follows. First, tools direct the work and the workers who use them, which makes tool-making a rich potential site for accessibility intervention and research into better understanding practitioner barriers when making and using tools. Second, the conversation between accessibility and data visualization is old, rich, and increasingly self-critical. Third, the field's output is lopsided: empirical breadth, exemplary prototypes, and guidance exist in abundance, while production tooling and practitioner action barely exist. The broader field's critical voices impose real constraints: tools do not solve access, and this thesis will not claim they do. Instead, critical work implores us to bridge the gap between research and practice. People with disabilities have been saying enough, it's time to get to work

and try to apply what we know at scale. Fourth, the gap is concentrated where practitioners' means run out in three specific ways: navigation lacks a structural substrate, interaction lacks input techniques for analysis, and personalization lacks infrastructure. Fifth, the first step in our thesis work should be to collect the empirical record and synthesize it for applied accessibility work in the context of interactive data visualization. This is Chapter 4's *Chartability* project. Given where the field stands, *navigation*, *input*, and *personalization* are the categories of work that follow Chapter 4; the chapters ahead carry this out as the body of this thesis, and our concluding chapters return to what tool-making accomplished and what it cannot accomplish, on the other side.

Chapter 3
Overview of Contributions

In this dissertation, we contribute practical advancements in tool-making as an intervention on the accessibility of interactive data experiences. The thesis of this dissertation is as follows:

This dissertation argues that the tools practitioners use to build interactive data experiences are themselves sites where accessibility barriers are produced, prevented, or alleviated for both end users and authors. This work contributes five tools, Chartability, Data Navigator, Skeleton, Cross-perception, and Softerware, that collectively center accessibility work on the empowerment of disabled and non-disabled practitioners across the full arc of evaluation, data navigation, analytical interaction, and personalization.

Rather than framing accessibility research solely around ideal experiences for end users with disabilities, this thesis investigates why accessibility work is so difficult for the practitioners who build interactive data experiences and what tool-making can reveal about those difficulties. We organize this investigation around four domains where practitioners face the most persistent challenges: evaluation, navigation, interaction, and personalization. *Chartability*, a heuristic framework, contributed first and mapped the full landscape of accessibility barriers. *Chartability* helped us to identify and prioritize our three latter domains (navigation, interaction, and personalization) as the areas where the most severe gaps remain in practitioner work. *Data Navigator* and *Skeleton* then investigate navigation, finding that practitioners struggle because navigation structure has no visible, manipulable representation in their workflows. *Cross-perception* engages interaction, demonstrating that blind data analysis has been constrained by existing tools and that a new interaction design framework can reshape what analytical work is possible. *Softerware* addresses personalization, revealing that access needs genuinely conflict across users and that meaningful personalization requires system-level infrastructure that does not yet exist in practitioner tooling ecosystems.

We engage each domain below with the questions: "Why does this work matter?", "Why is it hard?", and "What has tool-making within this domain showed us?"

The four domains of work that we engage in this thesis start first with **evaluation**. In work contexts where someone is designing and developing interactive data experiences, the practitioner must have the knowledge, tools, and resources available to systematically identify how their interfaces produce barriers for people with disabilities. A significant portion of professional accessibility work (arguably most, if not all) is founded on auditing and evaluating barriers to access. This work is pre-dominantly done through a standards-based approach [136], although in more robust evaluation work, people with disabilities are actively involved in the process [101].

Evaluation is difficult work because much of it is contextually defined by the author themselves, and most tasks at this intersection require careful, non-automated processes and meth-

ods [25, 101]. To make matters more difficult, no comprehensive guidelines, tests, and tools exist in any singular location. Practitioners often must gather these resources themselves, which tend to be situated towards accessibility in general or are high-level and provide minimal usefulness in practice. Additionally, practitioners themselves often have little knowledge about the veracity or quality of any given bit of information they gather [71, 119], and often do the work themselves to synthesize this disparate space of information into a usable format they can apply to their own evaluation work.

To engage this, our first main chapter focuses on *Chartability*, a heuristic framework that enables designers, developers, and auditors to systematically evaluate data visualizations and interfaces for a wide range of accessibility barriers, considering people with visual, motor, vestibular, neurological, and cognitive disabilities. In this project, we did the hard work for other practitioners and contributed our collection of synthesized accessibility resources in a single workbook. We had practitioners try out our resource in real environments, in-situ, in order to learn more about the challenges and barriers they themselves faced in evaluation work. With *Chartability*, practitioners, especially those with limited accessibility expertise, gained more confidence and clarity in assessing and improving their work. Additionally, *Chartability* has since become widely applied as a framework that isn't just used for evaluation but also as design guidance in many contexts, internationally, including policy organizations, governmental groups, and more than 100 companies and businesses.

Chartability then opened up a significant landscape of new projects and research directions. From a combination of my existing expertise as a visualization designer and engineer, in addition to our continued application of *Chartability* in the wild, we began to identify the trickiest and most-difficult domains of work for practitioners. Chartability has 50 total heuristics, or tests, each organized under one of seven principles. These three domains did not simply "emerge" on their own; they were evidenced by our applied work. Alongside the research activities that comprise this thesis, in a freelancing role we applied *Chartability* by partnering with industry, government, and non-profit organizations, conducting over 150,000 tests (both manual and automated) and working closely with dozens of designers and engineers. From that body of work, three larger domains stood out as the areas where the most severe and dramatic accessibility barriers remained unaddressed: data *navigation*, analytical *interaction*, and interface *personalization*.

So the next section of this thesis engages the first of these three: **navigation**. Navigation is a fundamental type of interaction that is leveraged by modern software-based assistive technologies. Screen readers, the primary tool used by people who are blind to interact with computers, navigate content. Additionally, many other assistive technologies, such as a sip and puff device (like the "POSSUM" from as far back as '63 [89]) also navigate. Navigational technologies are leveraged by people with a significant array of disabilities, yet tend to be entirely ignored by existing data visualization tools, which are pre-dominantly built to support direct input (using a computer mouse).

Empirical work has already demonstrated that structural navigation is actually good [157], even when regular alternative text (image descriptions) exist. This is both because people who are blind can gain both a high level understanding (from the description) as well as lower-level sense of the data's structure and arrangement, in addition to the fact that discrete, structural navigation exposes interactivity that may exist on any visualization elements (such as they can be hovered or clicked with a mouse in order to perform some action). So, if good empirical work exists: *why*

haven't practitioners put this research into action? What makes this work hard to do?

We first built *Data Navigator* to provide the building blocks we needed in order to address the technical and conceptual gaps that were required to make any visualization or visualization tool provide a navigable, interactive structure. *Data Navigator* is a low-level toolkit which can be used to construct accessible navigation structures such as lists, trees, and diagrams from an underlying graph structure. We leveraged graph theory for an applied HCI problem: nodes and edges represent any relationships within the data as a structure, which then supports rich expressiveness of data navigation experiences. Users can navigate discrete marks in a visualization, clusters, groupings, and more. In addition to its structural scaffolding, *Data Navigator* also supports a wide array of input modalities leveraged by people with disabilities (screen readers, keyboards, speech, and gestures).

Data Navigator provided a substrate, but this contribution alone wasn't enough to engage the question *why is navigation so hard, in practice?*. So the chapter following *Data Navigator* introduces *Skeleton*, a data navigation authoring tool built on top of *Data Navigator*. Our novel approach in *Skeleton* involves visualizing and making manipulable the nodes, edges, and textual data that comprise non-visual end user experiences. *Skeleton* visualizes the building blocks that comprise *Data Navigator*. Additionally, *Skeleton* provides expressive, rapid scaffolding capabilities that leverage data visualization rendering engines. This scaffolding engine helps practitioners quickly create common configurations for non-visual data navigation structures that retain visual congruence to the underlying structure.

But most importantly, *Skeleton* serves as a framework that shapes designerly consideration. Our conjecture was that because sighted practitioners cannot *see* navigation building blocks, they will not treat those elements as iterable design materials. We conjectured: Navigation is hard in practice because sighted designers face barriers to iteration and understanding. We conducted an empirical study with sighted practitioners and found that making non-visual elements visual helped practitioners shift from treating accessibility as a compliance task to treating it as a design problem, re-iterating on the visual aspects of their design, and engaging in the complex and nuanced components that comprise data navigation experiences.

Now, **interaction** becomes the next area we want to engage. Existing accessible data interaction for people who are blind, including our previous work on navigation (which is a form of interaction), predominantly seeks to expose information. This is what we call *access-oriented interaction*. In terms of low-level analytical tasks, most are then made feasible through navigation, sonification, or summarization-based and question-answering approaches: retrieving values, filtering, computing derived values, sorting, determining ranges, clustering, and finding outliers. What remains are analytical tasks that, despite being "low level" (understood as *unable to be reduced into more fundamental tasks*), are cognitively highly complex: finding correlations and characterizing distributions [3]. These tasks require complex hypothesization and exploration, rather than a system that simply encourages surfacing what is known or what is already present in the data: it requires combining, remixing, restructuring, and dividing data.

But blind *analytical interaction* isn't just important to engage because it is understudied, it is important to engage because many of interactive information visualization's most impactful tools for data science enable it [36, 54, 99, 142]. In visualization, *cross-filtering* is one example of an interaction that enables a user to filter one visual space while seeing a coordinated change in another visual space simultaneously and near-instantaneously. The speed of input interaction

and perception of output also matters: even a small bit of latency changes the quality of a user's data exploration activities [82]. We conjectured that a screen reader, the most-used tool leveraged by blind people when interacting with computers, may be insufficient for engaging this task.

To engage this, this thesis introduces *Cross-perception*, an approach for building analytical interactions that support perception in one space of input interaction with simultaneous, non-competing perception of output in another space of data representation. We first formalized a design framework for producing *cross-perception* experiences and then built a novel prototype device, the *cross-feelter*, that enables blind *cross-perception* of a cross-filtering data exploration interface. In an empirical study with blind users (with and without existing data expertise), we found *cross-perception* speeds up analytical exploration by 90% and helps blind users consider vastly more questions of their dataset (+188% computational queries, +54% spoken aloud) compared to a screen reader-driven interaction.

Beyond performance, we found that our input modality itself shaped the character of analytical engagement: participants didn't just work faster, they considered more dimensions of their data and asked qualitatively different questions. The *cross-feelter* also reduced anxiety and substantially increased enjoyment, particularly for participants without prior data expertise, suggesting that the barriers blind practitioners face in data work are not only functional but affective. Additionally, we had our blind practitioners imagine new interaction possibilities that *cross-perception* could enable including and beyond our *cross-feelter* device.

Our final domain of work is the most difficult for visualization practitioners to engage: **personalization**. While *navigation* demands better software tooling and visual support and *interaction* requires new hardware, *personalization* completely re-orients how software authoring takes place. Personalization matters because of *access friction*, which is a design challenge where one design or interface configuration that might be accessible for one person or group of people turns out to create barriers for someone else [49, 65]. In existing work on personalization and accessibility, studies have demonstrated that end-user control is great to have and can alleviate friction [70, 152], but little work has been done to explore what personalization looks like for an existing data visualization library and how practitioners should build and maintain their existing systems to support it.

Our final chapter introduces *Softerware* to address the tension between standardized accessible design and the diverse needs of real users with disabilities. In the wild, *access friction* exists in every design that reaches a public audience; it is inevitable. This tension ultimately means that some users have a worse experience, and may even face exclusion, with any particular design configuration. So for this work, we conducted our research in-situ with visualization software engineers and designers and worked to build a scalable, flexible software system dubbed "softerware" that enables end users to manipulate the appearance and functionality of the charts and graphs they encounter according to their own preferences. We conducted empirical research to inform our collaborators, as well as other visualization system authors, with guidelines and considerations for building *softerware* systems. In our study, no two participants chose the same preference configuration and participants with the same diagnosed condition sometimes needed opposite design treatments. Practitioners, meanwhile, immediately raised ethical concerns about whether personalization would let designers off the hook for poor defaults. These findings revealed that the real barriers to personalization are not at the level of any individual chart but at the level of system infrastructure: without persistence, cross-system interoperability, and shared

standards, the effort required of end users exceeds the value they receive.

Combined, these contributions provide empirical insights and practical advancements in the state of the art for tooling that bridges gaps in current accessibility practices in visualization and data science. Our work ultimately enables people with and without disabilities to better evaluate barriers in, analyze with, design for, develop, and personalize interactive data experiences. We demonstrate that tool-making is a productive intervention that both engages accessibility barriers and elucidates why those gaps exist in practitioner work.

Part III

Navigation: Making Data Structures Traversable

Chapter 5
Data Navigator: Low-level Tooling for Creating Navigable Data Structures

5.1 Preface

Making data visualizations accessible for people with disabilities remains a significant challenge in current practitioner efforts. Existing visualizations often lack an underlying navigable structure, fail to engage necessary input modalities, and rely heavily on visual-only rendering practices.

These limitations exclude people with disabilities, especially users of assistive technologies. To address these challenges, we present Data Navigator: a system built on a dynamic graph structure, enabling developers to construct navigable lists, trees, graphs, and flows as well as spatial, diagrammatic, and geographic relations. Data Navigator supports a wide range of input modalities: screen reader, keyboard, speech, gesture detection, and even fabricated assistive devices.

We present 3 case examples with Data Navigator, demonstrating we can provide accessible navigation structures on top of raster images, integrate with existing toolkits at scale, and rapidly develop novel prototypes. Data Navigator is a step towards making accessible data visualizations easier to design and implement.

5.1.1 Context

Data Navigator came about as a project largely motivated by my (Frank's) work in industry, at Visa working to make *Visa Chart Components* [135]. We had a top-down imperative to make our design system library of charts as accessible as possible, even pushing the limits of what that actually means. I led these efforts. It was clear to me from this work that one of the major technical barriers was enabling screen reader and assistive technology navigation of our charts, which is especially important when our charts were configured by our developer-users to have interactivity enabled.

At Visa, we rendered our charts using SVG (scalable vector graphics) in the browser. And it is possible to make SVG navigable to a screen reader by adding ARIA [139]. However, this approach only works for screen readers, meaning someone else who leverages a navigation-based assistive technology (such as a sip-and-puff, switch, keyboard, and so on) wouldn't have access. This was an incredibly difficult problem for us to solve at the time, especially since even ARIA's basic support and adoption varied significantly across devices and browsers at the time. Instead, I decided that we would programmatically render accessible, native HTML above our charts as someone navigated. For our uses at Visa, this approach worked.

Our approach had three major issues that blocked broader adoption: first, our work was tightly coupled to our internal visualization engine that rendered our chart components. It expected SVG. Next, the interactivity we built was *only* capable of perfectly mirroring whatever

interactions we had already built into our chart components for clicking and hovering actions. This limited which interactions were possible. And lastly, our structure wasn't grounded in any theory of data structures, but rather simply coupled to a navigation pattern we applied to nearly every chart in our chart library (with a few exceptions). And the exceptions in our library (for example, a circle-packing chart) made me realize we needed something about our navigation pattern to change fundamentally at a low-level to allow for broader experiences. Adding new features and refactoring existing ones was a brittle, slow experience.

For this project, we started nearly two years after I left Visa. Those lessons still sat in my mind. A better system was possible. I wanted to create a general system that any library rendering charts could use or replicate. Not to spoil one of the major intellectual foundations for this chapter, but we decided to use a graph as the substrate that scaffolds all other structures we might want for discrete navigation. By using a graph and creating a generic and low-level API, we created a tool that could express *virtually any structure*. Of course, we also simply de-coupled our structure from rendering methods entirely as well as de-coupled our interactivity from any set of assumptions about which interactions are possible for end users. These changes enabled *Data Navigator* to become what I like to call a set of "LEGO bricks" with enormous potential for solving existing problems and enabling new designs.

At the end of this chapter, we do a few things for the sake of this thesis: discuss the project in more technical detail and reflect on the successes that followed this publication.

This chapter was adapted from our published paper:

> F. Elavsky, L. Nadolskis, and D. Moritz, 'Data Navigator: An Accessibility-Centered Data Navigation Toolkit', *IEEE Transactions on Visualization and Computer Graphics*, pp. 1–11, 2023.

5.2 Overview

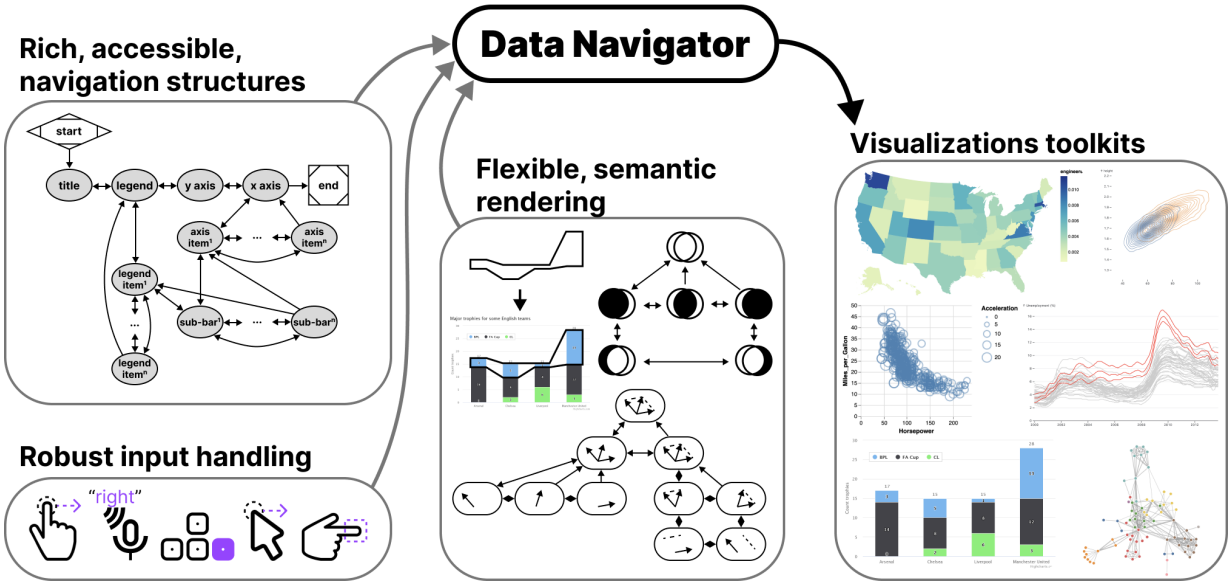


Figure 5.1: Data Navigator provides data visualization libraries and toolkits with accessible data navigation structures, robust input handling, and flexible semantic rendering capabilities.

While there is a growing interest in making data visualizations more accessible for people with disabilities, current toolkit and practitioner efforts have not risen to the challenge at scale. Major data visualization tools and ecosystems predominantly produce inaccessible artifacts for many users with disabilities. We believe this is largely a gap caused by a lack of underlying structure in most visualizations, failure to engage the input modalities used by people with disabilities, and over-reliance on visual-only rendering practices.

Users who are blind or low vision commonly use screen readers and users with motor and dexterity disabilities often do not use "pointer" (precise mouse and touch) based input technology when interacting with digital interfaces. Many users with motor and dexterity disabilities use discrete navigation controls, either sequentially using keyboard-like input, or directly using voice or text commands.

Most interactive visualizations simply focus on pointer-based input: they can be clicked or tapped, hovered, and selected in order to perform analytical tasks. This excludes non-pointer input technologies. These devices require consideration for the navigation structure and underlying semantics of a visual interface.

However, building navigable spatial and relational interfaces is a difficult task with current resources.

Raster images, arguably the most common format for creating and disseminating data visualizations, currently cannot be made into navigable structures. These are only described using alt text, which limits their usefulness to screen reader users.

Unfortunately, more accessible rendering formats like SVG with ARIA (accessible rich internet applications) properties are more resource intensive than raster approaches, like WebGL-powered HTML canvas or pre-rendered PNG files. SVG puts a burden on low-bandwidth users and a ceiling on how many data points can be rendered in memory.

In addition, ARIA itself has 2 major limitations. First, when added to interface elements, ARIA only provides *screen reader* access, which means that developers must build a solution from scratch for other navigation input modalities. Second, ARIA's linear navigation structure can be time-consuming for screen reader users if a visualization has many elements. This may impede how essential insights and relationships are understood [42, 72, 117, 123, 132, 157].

Some emerging approaches have sought to address this serial limitation of data navigation and provide richer experiences for screen reader users [42, 123, 132, 157]. However, these approaches rely on a tree-based navigation structure which is often not an appropriate choice for visualizations of relational, spatial, diagrammatic, or geographic data. Many visualization structures are currently unaddressed.

Zong et al. stress that in order to realize richer, more accessible data visualizations, the responsibility must be shared by "toolkit makers," the practitioners who design, build, and maintain visualization authoring technologies [157]. Our contribution is towards that aim, to make more accessible data experiences easier to design and implement within existing visualization work.

We present Data Navigator. Data Navigator is a toolkit built on a graph data structure, within which a broad array of common data structures can be expressed (including list, tree, graph, relational, spatial, diagrammatic, and geographic structures). Data Navigator also exposes an interface that supports interactions via screen reader, keyboard, gesture-based touch, motion gesture, voice, as well as fabricated and DIY input modalities. Data Navigator provides expressive structure and semantic rendering capabilities as well as the ability for developers to use their own, preferred method of rendering.

Data Navigator builds upon human-studies motivated work on accessible navigation [132, 157] towards a more generalizable resource for visualization practitioners. We contribute a high-level system design for our node-edge graph-based solution as well as an implementation of this system on the web, using JavaScript, HTML, and CSS. Through our case examples we also demonstrate that our generalized approach is suitable for replication of existing best practices from other systems, integration into existing visualization toolkit ecosystems, and development of novel prototypes for accessible navigation. We illustrate how Data Navigator's use of generic edges, dynamic navigation rules, and loose coupling between navigation and visual encodings provides practitioners robust, expressive control over their system designs.

5.3 Related Work

Our contribution is an attempt to bridge the gap between research and practice more effectively across broad ecosystems in order to enable deeper and more expressive accessible data navigation interfaces. Below we outline the prior research and standards that inform our project, a breakdown of existing visualization toolkit approaches to data navigation, and then accessible input device considerations.

5.3.1 Accessibility research and standards in visualization

Research and standards are both somewhat limited by a strong bias towards visual disabilities. In *Chartability*, 36 of the 50 criteria related to accessible visualization considerations involve visual disabilities [32, 33]. Marriott et al. also found that visual disability considerations are the primary focus of data visualization literature [92], leaving the barriers that many other demographics face unstudied.

However, despite the heavy focus on visual disabilities, the work that does exist in the visualization community is deeply valuable and serves as an important starting point for our technical contribution.

5.3.1.1 Accessible navigation design considerations

Zong et al.'s research, which was conducted as in-depth co-design work and validated in usability studies involving blind participants, presented a design space for accessible, rich screen reader navigation of data visualizations. They organized their design space into *structure, navigation,* and *description* considerations and demonstrated example *structural*, *spatial*, and *direct* tree-based approaches [157].

Chart Reader also engaged these design space considerations in their co-design work on accessible data navigation structures [132]. We consider these design dimensions as the best starting point for our work, bridging the gap between research and toolkits.

There are additional research projects that have focused on accessible data navigation and interaction [42, 116, 118, 123]. These contributions explore a range of different interaction structures, including lists, trees, and tables of information as well as direct access methods such as voice interface commands and simple, pre-determined questions.

5.3.1.2 Accessible visualization: understanding users

A wide array of emerging research projects investigate screen reader users' needs, barriers, and preferences, and offer guidelines, models, and considerations for creating accessible data visualizations [21, 33, 84, 117]. Jung et al. offer guidance to consider the order of information in textual descriptions and during navigation [72]. Kim et al. collected screen reader users' questions when interacting with data visualizations, which could open the door for more natural language data interaction [76].

5.3.1.3 Accessibility standards and guidelines

In the space of research, there has been a growing interest in developing guidelines for practitioners [30, 32] and even applying guidelines as a method of validation alongside human studies evaluations and co-design [33, 84, 85, 157]. Unfortunately, most accessibility standards and guidelines do not explicitly engage how to structure data navigation.

Despite this, existing accessibility standards bodies like the Web Content Accessibility Guidelines do stress the importance of accurate, functional semantics in order for screen reader users to know how to interact with elements [138]. For interactive visualizations this means that button-like or link-like behavior should expressly be made using elements that are semantically buttons and links. Our system should be capable of expressing meaningful semantics to users of assistive technologies.

5.3.2 Visualization toolkits and technical work

Unfortunately while many data visualization toolkits offer some degree of accessible navigation and interaction capabilities to developers, very few toolkits currently out there offer control over the important aspects of accessible data navigation design. Replicating existing research and strategies, remediating toolkit ecosystems, and building novel prototypes are all difficult or impossible to do due to the current lack of toolkit capabilities.

Existing data visualization toolkits have 3 major limitations that we wanted to address in the design of Data Navigator:

1. **Built on visual materials**: toolkits produce either raster or SVG-based visualizations, neither of which are focused towards designing navigable, semantic structures. As a consequence, many visualizations are simply entirely inaccessible.
2. **Lacking relational expressiveness**: When data navigation *is* provided, the navigation is based on either a tree or list structure (see Figure 5.2). The consequence of this limitation is that many other non-list and non-tree data relationships become difficult or impossible to represent without overly tedious navigation or inefficient architecture.
3. **Designed only for screen reader interaction**: When *accessible* data navigation is provided, it is generally only made possible through SVG with ARIA (Accessible Rich Internet Application) attributes. ARIA is primarily only leveraged by screen readers [139]. If a data element can be clicked and performs some form of function, only direct pointer (mouse and touch) and screen reader users are included. The consequence of this is that a wide array of other input devices, many used as assistive technologies by people with motor and dexterity disabilities, are excluded.

5.3.2.1 Rich, tree-based approaches

De-coupling rendered, visual structures from meaningful and effective navigation experiences can provide richer experiences for screen reader users [157]. Prior research and industry work, with the exception of the *Visa Chart Components* library [135], has relied heavily on a 1 to 1 relationship between structure (the encoded marks) and navigation. This emerging work is significant, because it paves the way for considering the design dimensions of accessible data

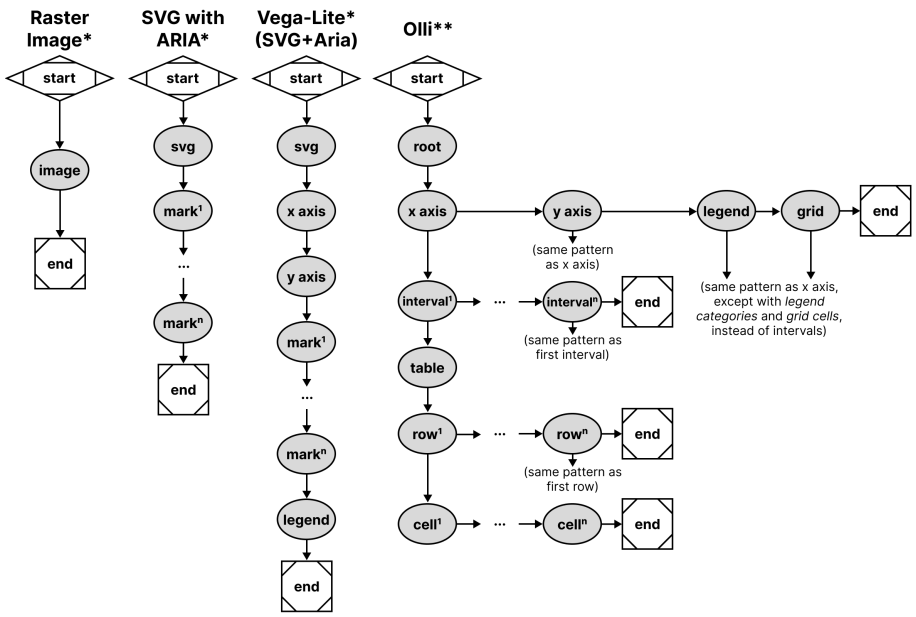


Figure 5.2: Existing accessibility trees and lists, shown using node-edge graph conventions. (*) Denotes only *screen reader* access. (**) Denotes *screen reader*, *keyboard-only*, and *pointer* access as well.

interaction and navigation without dependence on a visually encoded space.

Olli's approach has been to build ready-to-go adaptors that automatically build multiple tree structures for a few ecosystems (*Vega*, *Vega-Lite*, and *Observable Plot*) and is entirely uncoupled from a data visualization's graphics. Their approach renders navigable tree structures *underneath* a visualization.

Other than *Olli*, *Highcharts* [60], *Visa Chart Components*, and *Progressive Accessibility Solutions'* visualization toolkits [42, 123] also primarily provide tree and list navigation structures across all of their chart types. These toolkits render their structures *upon* the visualization's graphic space. These tools also provide some degree of support for other assistive technologies and input modalities, although are limited exclusively to SVG rendering.

Unfortunately, these toolkits lack capabilities for dealing with graph, relational, spatial, diagrammatic, and geographic data structures.

5.3.2.2 Serial, list-based approaches

Toolkits like *Vega-Lite* [111] and *Observable Plot* only provide basic screen reader support through ARIA attributes when visualizations are rendered using SVG. These libraries do not currently provide additional access to other assistive technologies and input modalities.

Microsoft's *PowerBI* largely uses a serial structure, although it has tree-like elements as well. *PowerBI* generally provides the same access to keyboard users as it does to screen readers, although not completely.

5.3.2.3 No navigation provided

Other visualization tools, like *ggplot2* or *Datawrapper*, *Tableau*, as well as both *Vega-Lite* and *Highcharts* (when rendering to canvas), produce raster images and have no navigable structure available. Raster, or pixel-based graphics have been an accessibility burden since the early days of graphical user interface development [16]. Practitioners who use these toolkits can only provide alternative text.

5.3.3 Considering assistive technologies and input devices

Modern data visualizations may contain functional capabilities such as the ability to hover, click, select, drag, or perform some analytical tasks over the elements of the visualization space [111]. Virtually all of these analytical capabilities are designed for use with a mouse.

Input device consideration can roughly be organized as either *pointer-based* (such as a mouse or direct touch) or *non-pointer based* (which may employ speech recognition or sequential, discrete navigation such as with a keyboard). Assistive pointer-based devices, such as a head-mounted touch stylus, can typically perform any actions that a mouse can and are therefore served by current interactive visualizations. However, assistive non-pointer devices, such as a tongue, foot, or breath-operated switch, are not.

By only providing pointer-based interactivity, modern interactive visualizations exclude users who leverage non-pointer based input, who are most commonly people with motor and dexterity disabilities. And unfortunately, there is a complete lack of engagement with these populations in the data visualization research community [92].

By comparison, the broader accessibility and HCI research communities have rich engagement with interaction and assistive technologies for users with motor and dexterity disabilities. Most research either focuses broadly on physical peripheral devices or sensors [122], wearables [109], or DIY making and fabrication [66].

The DIY making space involves a broad spectrum of complex input devices and materials, such as fabricating with wood and sensors for children with disabilities [81], 3D printed materials for rehabilitation professionals [40], and even using produce-based input (such as bananas and cucumbers) for aging populations [103].

Broadly, both research and practical developments related to accessible, non-pointer input are much further ahead than data visualization research and practice. Our goal for Data Navigator is to provide a technical resource towards engaging this under-addressed space.

5.4 Data Navigator: System Design

We categorized our system design goals into design considerations for *Structure*, *Input*, and *Rendering*:

1. **Generic structure and navigation specification**: Human studies work has validated that lists, tables, trees, and even pseudo-treelike and direct structure types are all valuable to users in different contexts and with different considerations. Our system must be able

to work with all of these as well as less frequently-used structures (spatial, relational, geographic, graph, and diagrammatic).
2. **Robust input handling**: Blind and low vision users may use combinations of different assistive technologies, such as magnifiers, voice interfaces, and screen readers. Users with motor impairments may rely on voice, gesture, eye-tracking, keyboard-interface peripherals (like sip-and-puffs or switches), or fabricated devices. Both the developer and user should therefore be able to leverage and customize a broad range of input types, including the above as well as fabricated, adaptive, and future input modalities.
3. **Flexible rendering and semantics**: Visuals may or may not be necessary to render to demonstrate Data Navigator's structure. In addition, much of the latest research has shown that different screen reader users may prefer different orders of information and at different levels of verbosity. In addition, the context of tasks the user is performing as well as the nature of the data itself may influence the design of semantic descriptions and visual indications for elements. Data Navigator must provide a high degree of flexibility and control.

To help bridge the gaps between research and standards knowledge about best practices and building an effective toolkit for practitioners, we intend for Data Navigator to provide both *exploratory support* and *vocabulary correspondence* [86].

In particular, our ideal users are developers who specify data visualizations using code. To that aim, we intend to provide *exploratory support* through generic, dynamic, and flexible system design decisions. Our system is expressive and customizable, which encourages exploration of different options.

And we also want the API to include properties that have conceptual and *vocabulary correspondence* to our design considerations. Each design consideration (*Structure*, *Input*, and *Rendering*) is separately composable, modular subsystems of Data Navigator that can be used independently or in tandem with one another.

In this paper we present an implementation of our system using JavaScript, HTML, and CSS on the web. The demonstration of our system is best suited to the web due to the nature of existing, accessible building blocks (HTML), which resolve many of the semantic complexities and logic involved in enabling screen readers to programmatically navigate and announce meaningful information to users. In addition, many existing visualization toolkits target the web as an output platform and we believe that this is the best starting point for adoption and use of Data Navigator. However, this system design could be implemented as a toolkit in other environments with proper consideration for input device handling and screen reader semantics.

5.4.1 Structure

5.4.1.1 Beyond trees: towards an accessibility *graph*

The first major contribution in the design of Data Navigator is to use node-edge data as the substrate for our navigation system.

The most important argument in favor of using a graph-based approach is that a graph can construct virtually any other data structure type (see Figure 5.2), including list, table, tree, spatial, geographic, and diagrammatic structures. Graphs are generic, which enables them to represent

structures both in current and future interface practices [39].

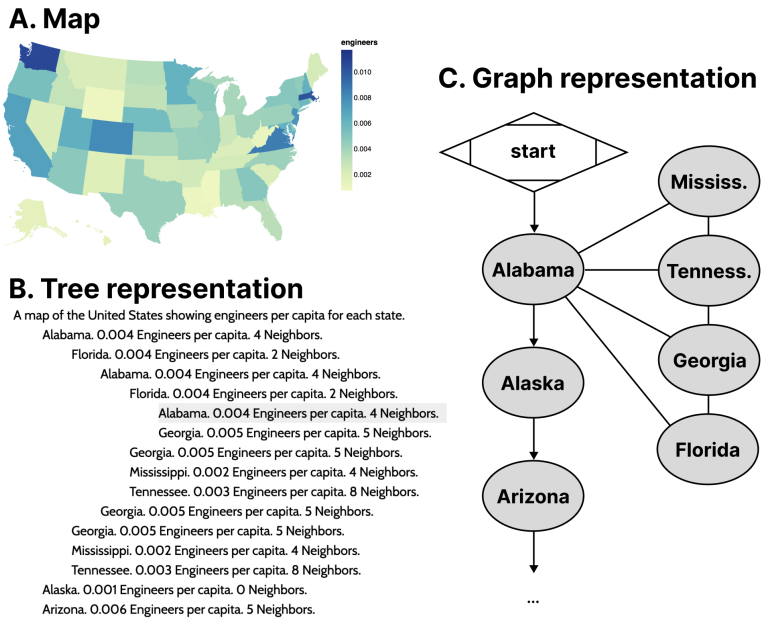


Figure 5.3: **A.** Map of engineers per capita of US states. **B.** Tree representation of the map data where states are listed alphabetically and also include links to neighboring states. The structure repeats itself if users navigate in a loop. **C.** Graph representation with the same navigation potential without redundant rendering.

To demonstrate our point, the most recent emerging work with advancements in accessible data navigation used node-edge diagrams to demonstrate their tree-like structures [132, 157] similar to Figure 5.2, Figure 5.9, and Figure 5.10. This is because trees are a form of node-edge graph, but with a root, siblings, parents, and children as sub-types of nodes that generally have rules for how they relate to one another.

Node-edge graph structures prioritize direct relationships. Examples of common direct relationships in visualization are boundaries on maps (see Figure 5.3), flows and cycles, data with multiple high level tree structures pointing to the same child datasets (such as *Olli* in Figure 5.2), or even just in diagrammatic, graph-based visualizations.

A graph structure allows for direct access between information elements that are not just part of the input data or 1:1 rendered elements, but may also have perceptual or human-attributed meaning. Examples of this might include semantic or task-based relationships, such as navigating to annotations or callouts, between visual-analytic features like trends, comparisons, or outliers. Spatial layouts such as intersections of sets or parallel vectors (see Section 5.5.3), or even relationships to information outside of a visualization and back into it (like in Figure 5.7) are enabled by a graph structure.

5.4.1.2 Graph structures are more computationally efficient

Data visualizations often portray information that becomes difficult to handle when using trees and lists. The distance users must travel between relational elements is significant in lists while redundancy when navigating relational elements in trees can be problematic.

As an example of this, often a data table or list of locations is used in conjunction with a map, such as listing all 50 states alphabetically along with relevant information. The list itself is expensive to navigate and may not provide any relationship information about which states border others, let alone ways to easily and directly access those states.

Part of the visual design justification of using a map instead of a table is for sighted individuals to understand how geospatial information may interact with a given variable. The spatial relationships matter. But when supplementing the list of states with sub-lists for each state's bordering states (see Figure 5.3), it produces redundancy in the rendered result. The rendered data contains circular connections between nodes but must render every reference, producing a computational resource creep and cluttered user experience that can be difficult to exit.

5.4.1.3 Specific edge instances and generic edges

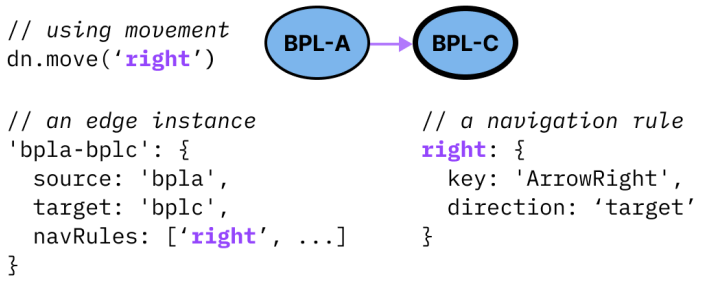


Figure 5.4: An example of how a single edge instance references a navigation rule and can even have multiple navigation rules. A navigation rule can be referenced by multiple edges.

In Data Navigator, nodes are *objects* that always contain a set of edges, where each edge contains a minimum of 4 pieces of information: a unique identifier, a source, a target, and navigation rules. These properties are only accessed when a navigation event occurs on a node with an edge that contains a reference to a rule for that navigation event. Navigation rules may be unique to an edge instance or shared among other edge instances.

The source and target properties of edges are either ids that reference node instances (see Figure 5.4) or *functions* (see Figure 5.5). Because some edges in a graph may be directed or not, non-directed graphs can use source and target properties to arbitrarily refer to either node attached to an edge.

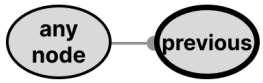
Generic functions for source or target properties can link nodes to other nodes based on changing content, structure, or behavior that may be difficult or impossible to determine before a user navigates the structure.

Function calling also allows some edges to be *purely* generic. An example of a reasonable use case of a purely generic edge is in Figure 5.5, where the source is a function which returns

the present node and the target is whichever node the user was on previously. This single edge may then be part of every node's set of edges, enabling users to have a simple *undo* navigation control without creating an *undo* edge unique to every source node.

Using this pattern, it is possible to have fully navigable structures using only generic edges.

```
// using movement
dn.move('previous position')
```




```
// a generic edge
'any-return': {
  source: (_d, current, _p) => current,
  target: (_d, _c, previous) => previous,
  navRules: ['previous position', ...]
}
```

```
// a navigation rule
'previous position': {
  key: 'Period',
  direction: 'target'
}
```

Figure 5.5: A generic edge, such as "any-return" can be applied to any node. Function calls handle dynamically assigning the edge's source and target nodes on-demand.

5.4.2 Input

5.4.2.1 Abstracted navigation facilitates agnostic input

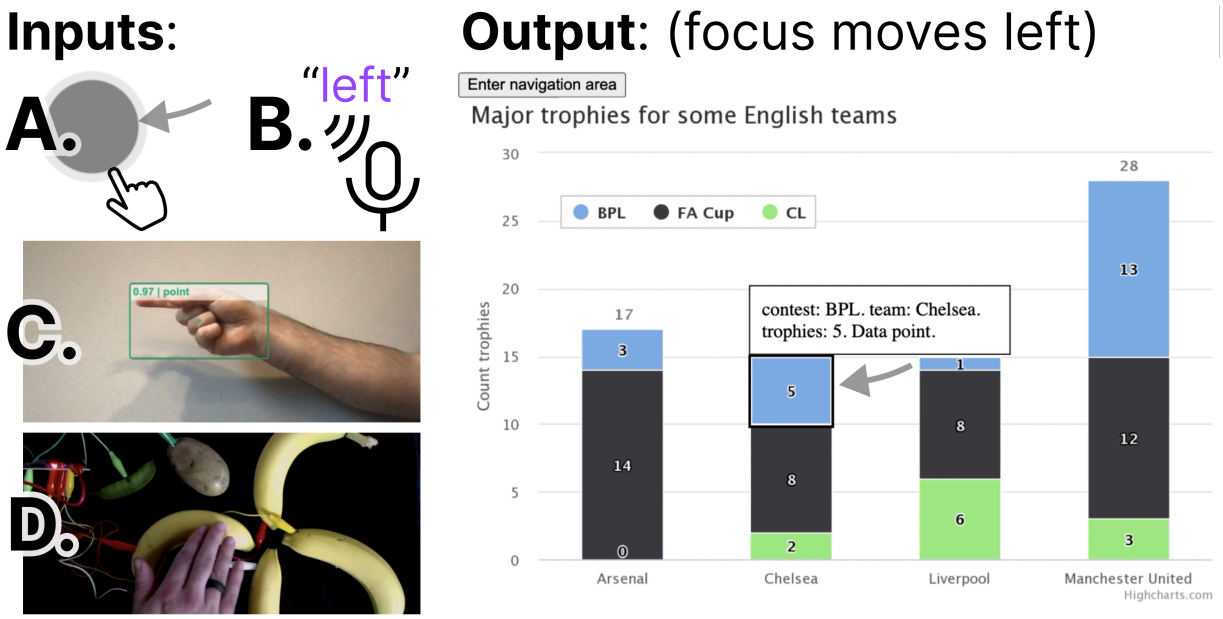


Figure 5.6: An example navigation rule to move "left" can be called as a method by an event from any input modality. Some examples include common modalities such as touch swiping (**A**) or speaking "left" (**B**). This also includes advanced or future modalities such as gesture recognition (**C**) or touch-activated, fabricated interfaces (**D**).

Navigation rules in Data Navigator (see Figure 5.4 and Figure 5.5) are created alongside the node-edge structure. Edges reference rules for navigation. However, these rules are generic and

agnostic to the specifics of input modalities and can be invoked as methods by virtually any detected user input event (see Figure 5.6).

Navigation rules are objects with a unique name, ideally as a noun or verb in natural language that refers to a direction or location, a movement direction (a binary used to determine moving towards the source or target of an edge), and optionally any known user inputs that activate that navigation, such as a keyboard keypress event name.

It is important for a system to abstract navigation events so that inputs can be uncoupled from the logic of Data Navigator. This allows higher level software or hardware logic to handle input validation while Data Navigator is just responsible for acting on validated input.

Later in our first case example (Section 5.5.1), we demonstrate an application that handles screen reader, keyboard, mouse and touch (pointer) swiping, hand gestures, typed text, and speech recognition input. Abstract navigation namespaces can be called by any of these input methods.

Additionally, since navigation rules are flexible, end users can also supply their own key-bind remapping preferences or input validation rules if developers provide them with an interface.

Because calling a navigation method is abstract, users can even supply events from their own input modalities as long as they have access to either a text input interface or access to Data Navigator's navigation methods. Our demonstration material (in Section 5.5.1) also includes handling for DIY fabricated interfaces, which are important in accessibility maker spaces. We chose a produce-based interface [103], since it was an easy and low cost proof of concept.

We believe that enabling agnostic input provides a rich space for future research projects. In addition, browser addons and assistive technologies could both leverage this flexible interface for end users.

5.4.2.2 Discrete, sequential input opens new avenues

The *keyboard interface* is considered foundational for many assistive devices, which leverage this technology for discrete, sequential, non-pointer navigation and interaction [140]. Desktop screen readers are the most common example of an assistive technology device that leverages the keyboard interface, however single or limited button switches, sip-and-puff devices, on-screen keyboards, and many refreshable braille displays do as well. Support for the keyboard interface by default in turn provides all discrete, sequential input devices with access as well.

However by basing Data Navigator's foundational infrastructure on a keyboard-like modality, this also provides designers and developers new avenues to imagine how existing direct, pointer-based, or continuous inputs can map to discrete, sequential navigation experiences.

For example, with mobile screen readers this already happens: screen reader users swipe and tap on their screen to sequentially navigate, but the exact pixel locations of their swiping and tapping generally does not matter. Their current focus position is discrete and determined by the screen reader software.

Data Navigator therefore allows for many new possibilities. One possibility is that sighted mouse and touch users may now also swipe their way through dense plots or use small interfaces (such as on mobile devices) that may otherwise be too hard to precisely tap. Data Navigator optionally removes the accessibility barriers sometimes posed by precision-based input in visualizations.

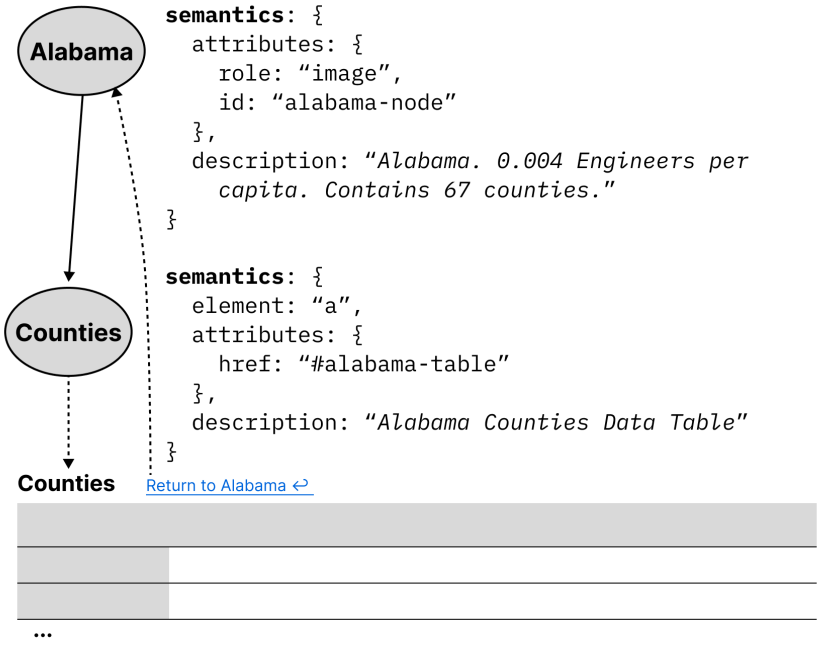


Figure 5.7: An example of how navigation within Data Navigator could use semantic nodes as hyperlinks to provide access to other areas in an application. Alabama has a child node “Counties” which is a semantic HTML link element pointing to a table of counties, outside of Data Navigator’s graph structure. A link is provided to return.

Data Navigator does not have to be in conflict with precision-based input, either. A discrete, sequential navigation infrastructure can be used in tandem with precision-based pointer events as well as instant access when coupled with voice commands and search features.

5.4.3 Rendering

5.4.3.1 Flexible node semantics provide freedom

Nodes in Data Navigator are semantically flexible. This is because the marks in a data visualization may represent many things, that are either dependent on the data or the user interface materials.

Since our toolkit implementation is in JavaScript and HTML, our map example from Figure 5.3 might use image semantics for states, alongside a description of the data relevant to that node. However since semantics are flexible in this way, Data Navigator could also be used to integrate into a larger ecosystem, with nodes rendered as hyperlinks to tables or other elements such as in Figure 5.7.

The concept of using node-edge graphs can even extend to have “nodes” that are entirely different parts of a document or tool, as well as integrated into the explicit structure provided by Data Navigator. In some accessibility toolkits, nodes are geometries without functional semantics [111] or list items nested within lists [13]. But in Data Navigator, nodes can semantically be buttons, links, or any HTML element. Interactive data visualizations sometimes demand more

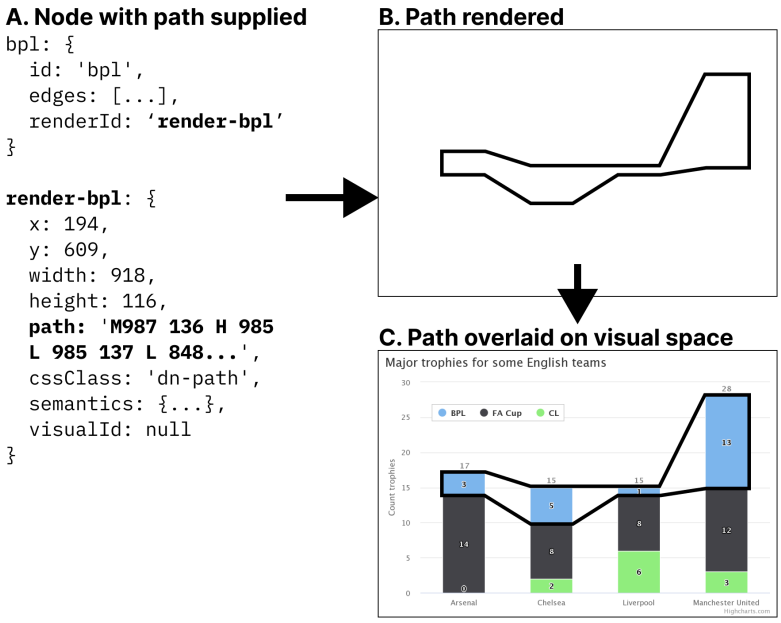


Figure 5.8: **A.** The data specified for a node with a reference to separate data that is used to render that node. **B.** The node will render as a path at the specified Cartesian coordinates. **C.** This rendered node may then be placed over a visual.

flexible node semantics than geometries or lists.

5.4.3.2 Loose-coupling to visuals enables expressiveness

One of the most significant technical limitations of existing data visualization toolkits with regards to accessibility is that they rely on visual substrate, or visual materials, in order to produce data visualizations. In the case of static, raster images such as png files or WebGL and canvas elements on the web, there are no interface properties at all exposed to screen readers for programmatic exploration and interaction.

If raster images are used, they generally cannot be changed after rendering. However, according to web accessibility standards, elements must have a visual indicator provided when focused [137].

Since Data Navigator navigates using focus, an indicator must be rendered alongside the node semantics. But *what* is focused visually and *where* it is depends on different design needs.

In *Visa Chart Components*, chart elements can be *selected*, so the focus indication is visible over the existing elements in the chart space. The design choice to have interactive visual elements located within a chart or graph is also common in other toolkits that provide accessible focus indication, such as *Highcharts*, *PowerBI*, and *SAS Graphics Accelerator*.

However, some visualization toolkits create accessible structures entirely uncoupled from visual space [13], so focus indication is provided beneath or beside the chart, not over it.

Due to the different ways that accessibility might be provided, Data Navigator enables developers to have complete control over the rendering of which focus elements they want, in what

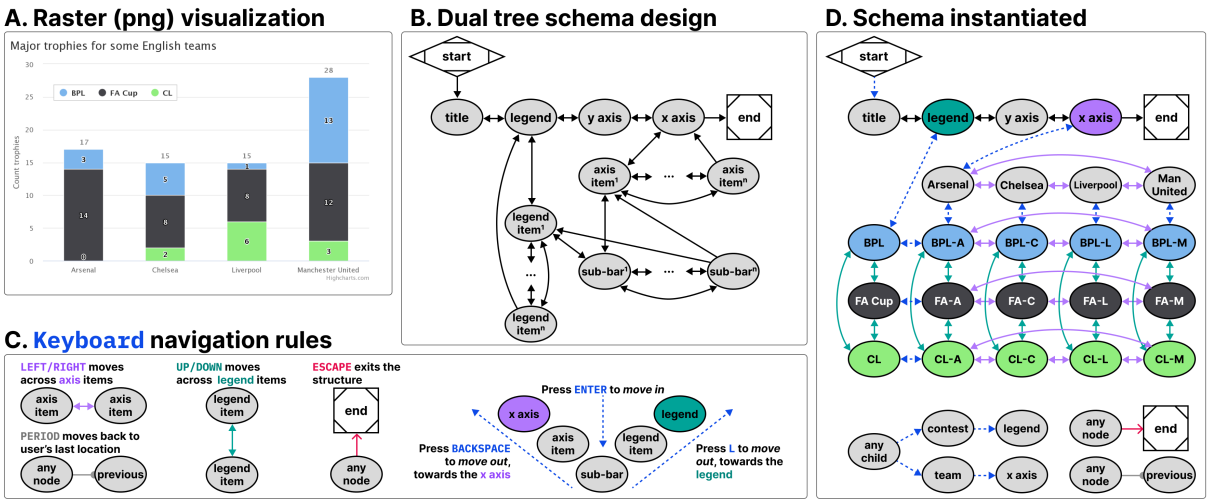


Figure 5.9: **A.** A raster (png) visualization of a stacked bar chart showing how 4 English teams performed across 3 major trophy contests. **B.** An example navigation schema that allows children nodes to have 2 parents (two tree structures intersecting), one for contests and one for teams. **C.** An example of Data Navigator's navigation logic abstraction, which allows edge types to have programmatic sources, targets, and rules, such as a single rule that gives all nodes an edge to exit the visualization. **D.** An instantiation of the schema, showing all corresponding rendered nodes and their edge types according to the schema design and navigation rules.

styling, and where. This can accommodate both un-coupled and visually-coupled approaches to focusing and more.

Data Navigator's focus is *uncoupled* by default and may even be used independent of any existing graphics at all. Rendering information may be passed to Data Navigator for it to render (like in Figure 5.8) or developers can provide their own rendered elements and simply use Data Navigator to move between them.

Because of Data Navigator's approach to rendering focusable elements, designers and developers can provide fully customized annotations, graphics, text, or marks that may not be part of the original visual space or elements. One example of this might be adding an outlined path to a collective cross-stack group of bars in a stacked bar chart (see Figure 5.8).

Loose-coupling in this way provides robust flexibility to designers and developers to handle navigation paths and stories through a data visualization, even in bespoke or hand-crafted ways.

5.4.3.3 On-demand node rendering is efficient

Practitioners care about performance and so do users. Practitioner toolkits often focus on lazy-loading techniques where accessibility elements are rendered on-demand rather than all in-memory up front [13, 31, 157].

Data Navigator's nodes are rendered *on-demand* by default. Data Navigator only renders the node that is about to be focused by the user and after it is focused, the previously focused node is deleted from memory. This technique has advantages in cases where datasets are large or users

have lower computational bandwidth available. However, there are cases where practitioners may want to render all of Data Navigator's structure in memory, such as server-side rendering or equivalent. Pre-rendering may be optionally enabled.

5.5 Case Examples with Data Navigator

We built example prototypes using our JavaScript implementation of Data Navigator, available open source at our GitHub repository.

Our first two prototype case examples represent some of the most powerful parts of Data Navigator as a system while reproducing known and effective data navigation patterns from existing industry and research projects. We provide a final case example as a co-design session that demonstrates how Data Navigator may be used to rapidly build new designs.

5.5.1 Augmenting a Static, Raster Visualization

The first case example (shown in Figure 5.9) builds on an online JavaScript visualization library, *Highcharts*. *Highcharts* already provides relatively robust data navigation handling out of the box for screen reader, keyboard, and even voice recognition interface technologies, such as *Dragon Naturally Speaking*. However, these capabilities are only provided when the chart is rendered using SVG. Developers have several other rendering options available, including WebGL, which is significantly more efficient [61]. We wanted to demonstrate that Data Navigator can provide a navigable data structure even if the underlying visualization is a raster image.

For our case example, we exported a png file using the built in menu of a sample stacked bar chart retrieved from their online demos [59]. We selected a stacked bar chart because it allows us to demonstrate how two tree structures may interact and share the same children nodes.

We recorded the data and hand-created all of the geometries and their spatial coordinates using *Figma*, by tracing lines over the raster image's geometries (see samples of the data and traced geometries in Figure 5.8). While this method was efficient for building an initial prototype, Section 5.5.2 engages deterministic methods for extracting and producing the nodes, edges, and descriptions required by Data Navigator automatically and at scale.

The visualization we selected represents 4 English football teams, *Arsenal, Chelsea, Liverpool,* and *Manchester United* and how many trophies they won across 3 contests, *BPL, FA Cup,* and *CL*.

We chose a schema design that arranged the *contests* to be navigable across one dimension of movement (*up* and *down*) while the *teams* are navigable across a perpendicular dimension of movement (*left* and *right*). This 2-axis style of navigation is used by *Highcharts* (when rendering as SVG) and *Visa Chart Components*. We also chose these directions because it is coincidental that their visual affordance is closely coupled with the navigation design (the x axis is ordered *left* to *right* and since the bars are stacked, *up* and *down* can move within the stack). These directions can also be applied to the axis categories and legend categories as well, moving *left* and *right* across the entire *team's* stacks or up and down across the entire *contest's* groupings.

Using a keyboard, a user might enter this schema and navigate to the legend, where they could press *Enter* to then focus the legend's first child, pictured in Figure 5.8. Pressing up or

down navigates in a circular fashion among the *contest* groupings. Pressing *Enter* again then focuses the first child element of that *contest*, all of which are in the *Arsenal* group, since it is the first group along the x axis. A user can then navigate *up, down, left,* and *right* among children. Pressing *L Key* moves the user back up towards the contest while pressing *Backspace* moves the user up towards the x axis. The x axis and *team* groupings represent the second tree which intersects the first (the *contests*).

Our first case example includes handling for additional input modalities beyond screen readers and keyboards, including a hand gesture recognition model, swipe-based touch navigation, and text input (which can be controlled using voice recognition software).

5.5.1.1 Discussion

Our first case example demonstrates several of the most important capabilities of Data Navigator, namely that practitioners can add accessible navigation to previously inaccessible, static, raster image formats and that a wide variety of input modalities are supported easily.

Widely-used toolkits like *Vega-Lite*, *Highcharts*, and *D3* [14] allow practitioners to choose SVG and canvas-based rendering methods. Data Navigator's affordances help overcome the lack of semantic structure in canvas-based rendering, allowing developers to take advantage of its processing and memory efficiency.

Notably in addition to these capabilities, the visual focus highlighting added was entirely bespoke (as in Figure 5.8) and the navigation paths through the visual were based on our design intentions, not an extracted view or underlying architecture such as render order. This demonstrates that our system provides a significant degree of freedom and control for designers and developers.

As a final discussion point, the resulting visualization contains no automatically detectable accessibility conformance failures according to the W3C's Web Accessibility Initiative's accessibility evaluation tool, *WAVE* [143]. It is important for any technology developed to also meet minimum requirements for accessibility [32, 84, 85, 157], even when following best-practices and research.

5.5.2 Building Data Navigation for a Toolkit Ecosystem

Our second case example, shown in Figure 5.10, builds on *Vega-Lite*. As shown in Figure 5.2, *Vega-Lite* offers basic screen reader navigation but provides no navigation at all when rendered using canvas.

While it might be a tedious design choice to allow every mark in a visualization to be serially accessible to screen reader users, we nevertheless set out to build a generic ingestion function that would take a *Vega-Lite View* object and deterministically recreate their existing SVG navigation structure in Data Navigator. This way users would have the same experience between SVG rendered charts and all current and future rendering options that *Vega-Lite* offers to developers.

Notably, *Vega-Lite* does not explicitly manipulate the navigation order at all when rendering with SVG. ARIA is simply provided to allow screen reader users to access each mark in the visualization in the order the mark appears in the DOM (which is the order it was rendered). The legend appears after the marks in our schema for this reason because *Vega-Lite* renders the

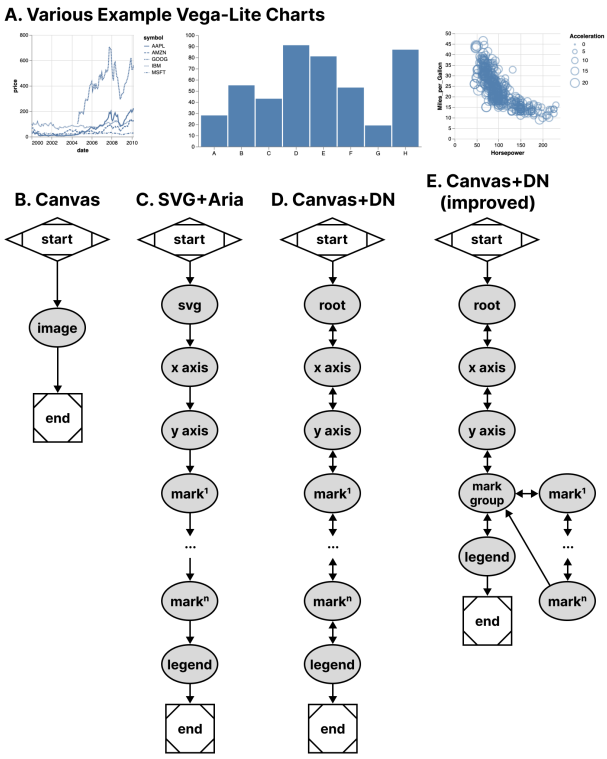


Figure 5.10: **A.** Various charts from *Vega-Lite* share the same general structures with each other when rendered using canvas (**B**) or SVG (**C**). **D.** With Data Navigator, we replicated the existing SVG navigation pattern (**C**) but used a canvas-based rendering for the visualization. **E.** We also improved the navigation scheme to nest marks within a mark group to allow users to skip them, if needed.

legend after marks. This choice of ordering is for visual reasons: z-axis placement is currently based on render order in SVG and *Vega-Lite* wants their legend visually on top of the rendered marks.

In addition to mimicking their existing SVG navigation strategy, we also created a way to nest all of the marks within a group so that users can skip past them and drill in on-demand, which is a valuable pattern when dealing with situations where providing a mark-level fidelity of information may not be relevant to a user's needs by default [121, 157].

In order to deterministically supply Data Navigator with accurate information about any given *Vega-Lite* visualization, we built 3 functions: one that takes a *Vega-Lite View* as input and extracts meaningful nodes, one that produces edges based on those nodes, and one to describe our nodes in a meaningful way for screen reader users. These generic functions technically work on all existing *Vega-Lite* charts, however some are more useful out of the box than others due to the type of marks involved.

5.5.2.1 Discussion

This case example demonstrates that ecosystem-level remediation and customization is not only possible for toolkit builders but Data Navigator offers robust potential. Data Navigator's structure, input, and rendering capabilities are all flexible and can be adjusted to suit the needs of a specific toolkit's design and intended use.

Many visualization libraries may not even provide screen reader accessible SVG using ARIA-based approaches but do have a consistent underlying architectural pattern. Some libraries have a consistent method for converting data into visual formats, readable text labels, and interaction logic. Strong contenders would be visualization libraries popular in online, web-based data science notebooks like *ggplot2* in R or *matplotlib* for Python, which typically only render rasterized pngs or semanticless SVG.

Toolkits with consistent underlying architecture would allow toolkit developers, not just developers who *use* toolkits, to remediate and customize their navigation accessibility using a generic approach.

Enabling accessibility at the toolkit level allows all downstream use of that tool to have better defaults, options, and resources available for building more accessible outcomes for end users.

Many libraries and toolkits provide users with a level of functional defaults and abstract conciseness so that users don't have to worry about low-level geometric considerations [111].

Data Navigator allows toolkit developers to also provide their users with abstractions and defaults for accessibility that make sense for their ecosystem.

Despite our schema recreating a screen reader experience based on SVG (and improving it), Data Navigator's additional features also apply: users are able to leverage a much wider array of input modalities.

Vega-Lite provides many ways to make marks clickable and even perform complex actions using mouse-based input. While Data Navigator does not engage accessible brush and drag-based inputs, it does provide keyboard-only access by default, which can be used to make events previously only accessible to mouse clicking available to many other technologies. This is an improvement over *Vega-Lite*'s SVG + ARIA rendering option.

When measuring performance across test datasets containing 406 and 20,300 data points in a scatter plot, Data Navigator increases initialization time by $\sim$0.45 to $\sim$1.5ms respectively. Our extraction functions specific to *Vega-Lite* increase initialization between $\sim$4.8 and $\sim$8.5ms respectively. Given that our benchmark testing for *Vega-Lite's* SVG rendering initialized in $\sim$1,800ms for 20,300 data points and canvas in $\sim$700ms, we do not anticipate that Data Navigator will have a negative impact on performance in most visualization contexts.

5.5.3 Co-designing Novel Data Navigation Prototypes

Recent projects in accessible data navigation have involved extensive co-design work with people with disabilities, ranging on the magnitude of months with as many as 10 co-designers at a time [84, 85, 132, 157].

However many visualization experiences may be authored in smaller scales, with fewer designers, and less time such as the development of a prototype or demonstration of an emerging idea. In practical or industry contexts, co-design sessions (and design sessions in general) may

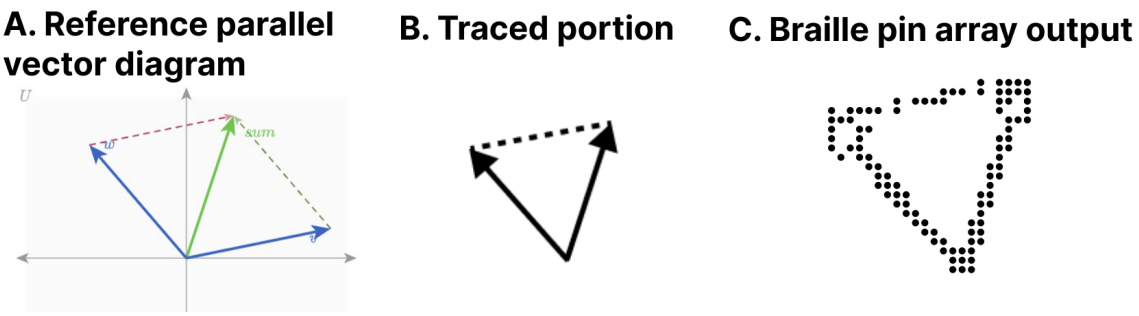


Figure 5.11: Our material preparation process involved taking a reference (**A**), tracing it (**B**), and rendering it on a tactile display (**C**).

be much shorter. The goal of these co-design sessions is simply to create an artifact with the artifact's intended users.

Since our paper is a contribution towards practical outcomes, we simulated a light co-design session with the aim of producing low-fidelity prototypes of novel data interaction patterns.

5.5.3.1 Co-design Session Methods and Setup

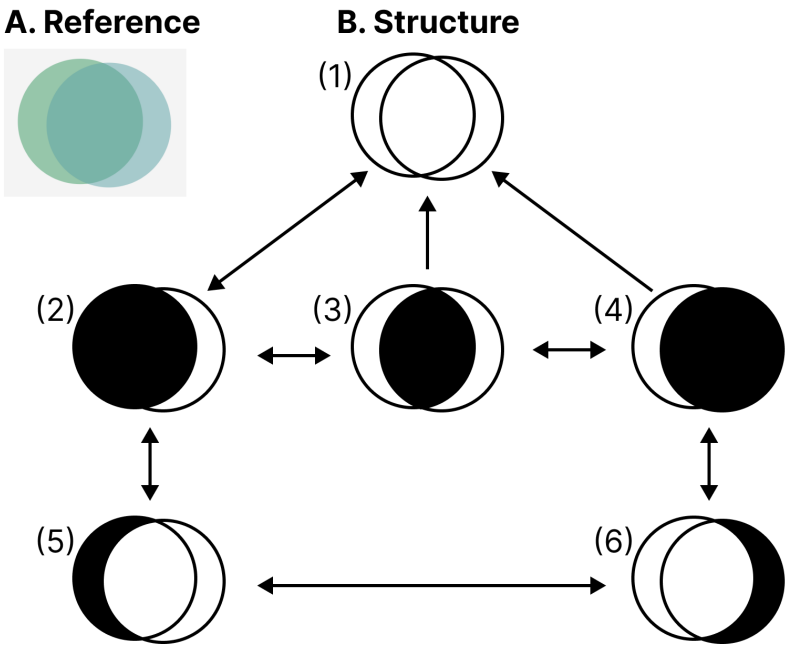


Figure 5.12: **A.** A reference image from *Penrose* of a set diagram containing two sets intersecting. **B.** A diagram of our proposed structure, with three levels of information.

Authors Frank Elavsky (sighted) and Lucas Nadolskis (blind) set out with the goal of developing screen-reader friendly prototypes that can explore geometric and mathematical models produced by the math diagramming tool *Penrose* [153].

Nadolskis is a neuroscience engineer who is a native screen reader user and uses both mathematical concepts as well as data-related tasks in his research. Elavsky proposed a series of possible math-based visualization types produced by *Penrose* to build prototypes for, and Nadolskis selected *set* and *vector* diagrams as the two worth exploring first. The justification for this selection is that understanding these two concepts is important for work in data science, programming, and more advanced math concepts.

In particular we grounded the context of our contribution in a hypothetical classroom setting, where a screen reader user who is a student will have access to the equations in both raw text and *MathJax*. We want to provide an experience that does not replace the existing resources screen reader users have to learn in classrooms but rather supplements them.

At our disposal for our co-design session was a *Dot Pad* [27], which is a refreshable tactile braille display. Our *Dot Pad* enabled Elavsky to produce something visual and then translate it into the display for Nadolskis. Similar to de Greef et al. [24], we used a tactile interface as an intermediary to help us get a shared sense of the meaningful spatial features of our figures.

Elavsky started with a reference diagram and then traced every possible node that might be worth navigating to in the diagram (see Figure 5.11).

We selected which nodes were most important in each diagram, how to navigate between them, and how we wanted to render their visuals and semantics.

The selection of our problem space, scope of solutions, context of contribution, general discussion, and preparation of materials took approximately 12 hours of work over 2 weeks. The exploration of our prototype design space for our 2 prototypes took 1 hour. Building the prototypes took 2 hours.

5.5.3.2 Creating a Navigable Set Diagram

Our first prototype was a set diagram (see Figure 5.12). For our structure, we decided that it has 3 important semantic levels: the high level, the inclusion level, and the exclusion level. The inclusion level is first and the siblings are all sets or subsets that include other sets. The exclusion level is beneath and contains sets or subsets that are exclusive to the sets they belong to, which are accessed by drilling down from a set.

Our schema design starts with a user encountering the root level (1) and may optionally drill in to the first child of the next level (2) using the *Enter* key. The user may navigate siblings at this level using *right* and *left* directions, but this level is not circular (like in Figure 5.9) to maintain the spatial relationships. The user may drill in on either set again to view the non-intersecting portion of that set. Any node can drill up, towards the root, using *Escape* or *Backspace*.

5.5.3.3 Creating a Navigable Parallel Vectors Diagram

Our second prototype was a parallel vectors diagram (see Figure 5.13). For the structure of this diagram we created a first level group that contains each vector and vector sum. The sibling to this grouping is another group which organizes sub-equations related to calculating each parallel vector. The sub equations each contain children that pair the sub equation with the vector it is parallel to.

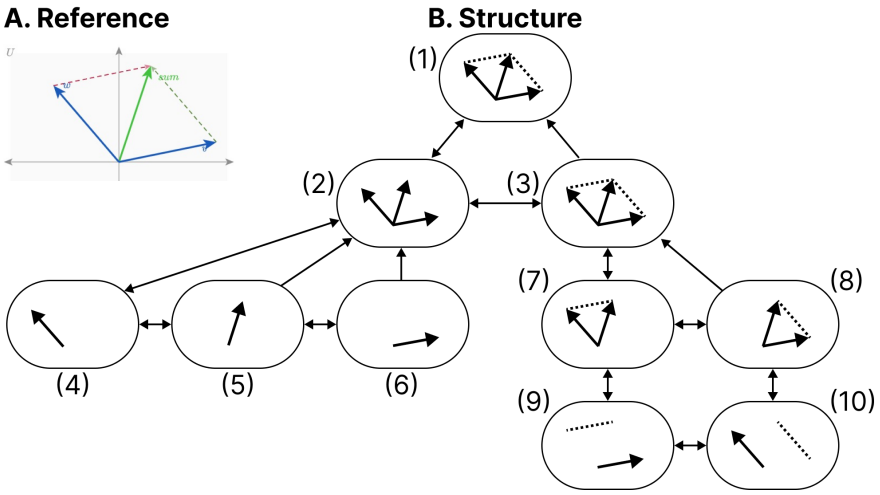


Figure 5.13: **A.** A reference image from *Penrose* of a parallel vectors diagram. **B.** A diagram of our proposed structure, with two main sub-categories of information: understanding the vectors and their parallels.

Similar to Figure 5.12, this figure maintains spatial relationships along the x dimension, does not have circular navigation, and allows drilling in and out.

5.5.3.4 Discussion

After our co-design sessions, our visual materials and navigation structures were used in the creation of functional prototypes. We additionally hand-crafted the descriptions and semantics for each node.

Accessibility work often takes a long time, from co-design to building to validation. But we believe that a well-articulated and useful design space, with tools that provide expressiveness and control over the dimensions of that design space, can improve how this work is done. The above case example demonstrates how builders who are thinking about data navigation design can rapidly scaffold prototypes for use in Data Navigator.

In particular, Data Navigator's design as a system gave our co-design sessions *vocabulary correspondence*. Data Navigator's language helped us focus on the *nodes, edges,* and *navigation rules* for our *structure* while we also explicitly discussed the *rendering* details of *coordinates, shapes, styling,* and *semantics* for each node. The vocabulary of our design space directly corresponded with code details required to create a functional prototype.

We note that this co-design work is not intended to contribute a *validated* set of designs. Rather, our contribution with this case example is to demonstrate that within the larger ecosystem of a research venture, Data Navigator is an improvement over designing and building navigable structures from scratch.

5.6 Limitations and Future Work

Data Navigator is a technical contribution, a system designed for appropriation [28] and adaptation [150] in different applied contexts. It is, as Louridas writes, a *technical material*: a technology that enables new and useful capabilities [83]. While beyond the scope of the current paper, a critical next step for future work is to conduct separate studies with both practitioners and end users to evaluate Data Navigator's affordances.

Unlike toolkits that provide an end-to-end development pipeline for accessible visualization, Data Navigator serves as a low-level building block or material (like concrete). As such, one potential limitation of the framework is that it can be used to build both curbs (which are inaccessible) as well as ramps and *curb-cuts* (which may be more broadly accessible).

Even when building more accessible curb-cuts, we stress the importance of actively involving people with disabilities in the design and validation of new ideas, in line with prior work [84, 85, 102, 157]. For example, while our first two case examples replicate co-designed and validated existing work, our third case example's co-designed prototypes would need to be validated with relevant stakeholders before wider implementation. Our system does not *guarantee* any sort of accessibility on its own.

The diverse array of modalities supported by Data Navigator opens an immediate line of future work in engaging people with a correspondingly diverse set of disabilities. While recent explorations into accessible data visualization have been inspiring, this trend has primarily focused on the experiences of people with visual disabilities [32, 77, 92]. More research should be conducted with other populations, particularly people who leverage assistive technologies beyond screen readers, to understand how interactive data visualizations can be better designed to serve them.

Finally, there are significant opportunities to improve the efficiency of our approach, including developing deterministic and non-deterministic methods to generate node-edge data and navigation rules from a visualization. Ma'ayan et al. stress in particular that reducing tedious complexity can contribute to the success of a well-designed toolkit [86]. Future work should identify areas where graphical interface tools or higher-level specifications can improve the experience of working with Data Navigator.

5.7 Conclusion

Practitioners at large continue to produce inaccessible interactive data visualizations, excluding people with disabilities. We believe that the burden of remediation first starts with the developers who build and maintain the toolkits that practitioners use.

However, the challenges faced by toolkit builders are significant. Most toolkits lack an underlying, navigable structure, support for broad input modalities used by people with disabilities, and meaningful, semantic rendering.

To engage these limitations we present Data Navigator, a technical contribution that builds on existing work towards a more generalizable accessibility-centered toolkit for creating data navigation interfaces. Data Navigator is designed for use by practitioners who both build and use existing toolkits and want a tool to make their data visualizations and interfaces more accessible.

We contribute a high-level system design for our node-edge graph-based approach that can be used to build data structures that are navigable by a wide array of assistive technologies and input modalities. Data Navigator is generic and can scaffold list, tree, graph, relational, spatial, diagrammatic, and geographic types of data structures common to data visualization.

Our system is designed to encourage both remediation of existing inaccessible systems and visualization formats as well as help scaffold the design of novel, future projects. We look forward to further research that explores the possibilities enabled by Data Navigator.

5.8 Postface

We unpack this in the next chapter, but our work on *Data Navigator* was never intended to end as a research prototype. We carefully engineered *Data Navigator* to focus on what we believed would be the most-necessary modules required for integrating into any web-based visualization library. Modern libraries render in many different ways, as SVG, HTML, canvas elements, and canvas built with WebGL under the hood. The actual work of rendering is offloaded onto a wide range of ecosystems, from React, to Svelte, to D3, to vanilla JS. Our *rendering* module was intended to work with any of it (or be entirely optional). This, we believed, separated us from every other research prototype the most. *Olli*, while a fantastic project, didn't have an independent rendering engine that could de-couple from the data structures it produces, same with *VoxLens* and other similar projects. These projects handle their own rendering: they produce elements that live on the DOM. Infrastructurally, this can be a problem for many teams when integrating into their production environments because the structure *is* the rendering.

While it certainly isn't hard to imagine how to make a de-coupling possible with those existing research prototypes, doing that work after their papers have been published is typically an additional burden on a research team that isn't actively incentivized. Maintenance and feature improvements are hard to justify when novelty drives technical research production most of all. These systems ultimately must be refactored to work with different rendering methods (SVG/canvas, etc) and rendering frameworks (React, D3, and so on). We intended to face this speedbump from the start and mitigate future refactoring as much as possible.

And in our next chapter, our work with *Adobe* wouldn't have been possible without this de-coupling approach with our rendering module. Coincidentally, the main approach that libraries like *Olli* and *VoxLens* must take is building couplings, wrappers, and pipelines between a separate space where their tech exists and a visualization exists. The spaces aren't integrated neatly. Accessibility exists in one area of an interface while visualizations are in another, and they must be managed separately. But in our work with *Adobe*, *Data Navigator* is part of the lowest-level processes that construct a chart experience, working in tandem with the specifications for their visualization engine. *Bokeh*'s ecosystem, on the other hand, made more sense to remain de-coupled to avoid placing future maintenance in the hands of the open-source *Bokeh* contributors.

And we didn't invent this notion. Vega technically just takes a specification and it produces a view model. How that model is actually turned into visible elements is up to the ecosystem [111]. But for accessibility, there tends to be a philosophy to tightly couple the rendering methods to the access model. We conjecture that this might simply be because many accessibility barriers are a result of an ecosystem's failure to use the right substrate. So an accessible software system *is* one that writes to a substrate.

What we might learn from *Data Navigator* then is that more accessibility projects should seek to abstract out the generation of accessible models and substrates from the means to enact those substrates. The precedent for this in visualization has empowered a single spec like Vega to integrate into the web [111], Python [134], and Rust [79], to name a few. And while *Data Navigator* was instantiated as a JavaScript library, the abstraction of the system as it was designed could lead to implementations in virtually any software environment.

Additionally, because this thesis can afford additional space, we are expanding here beyond the already-published chapter and unpacking some more details about *Data Navigator*, techni-

cally speaking. Every subsequent chapter will also have additional technical details included (beyond their published papers), to help archive and document more about how these systems were made.

5.8.1 The library boundary: what ships, and what demos build

Before describing each subsystem, it is worth being precise about what Data Navigator the *library* provides and what our case examples build on top of it. Previously, this chapter differentiates between the system and the instantiation of our system. Here, we refer to the current-working instantiation of Data Navigator as our *library*. Data Navigator's library is distributed as an npm package, `data-navigator`, written in TypeScript and compiled to both ES module and CommonJS formats with type declarations. Installing the package provides three composable modules, one for each of our design considerations: `dn.structure()`, `dn.input()`, and `dn.rendering()`. These can be used independently or in tandem. The package also ships a small set of supporting utilities, including `describeNode()` for generating default text descriptions from a datum, `createValidId()` for sanitizing identifiers, and a collection of pure geometry functions (convex hulls, unions of rectangles, and bounding outlines) used to compute focus outline paths for groups of elements.

The three modules divide responsibility as follows. The *structure* module produces the graph: given either hand-authored node and edge objects, a flat dataset with a declared set of dimensions, or a *Vega-Lite* view object, it returns nodes, edges, and the navigation rules that edges reference. The *input* module is a pure traversal engine: given a structure and its rules, it answers the question "from this node, where does this navigation command lead to in the structure?" The *rendering* module maintains a semantic HTML layer: a wrapper with appropriate ARIA properties and focusable elements positioned over (or beside, or independent of) the host visualization. The implementation details of each module are described at the end of their respective subsections below.

Just as important is what the library deliberately does *not* do. Data Navigator does not render charts, does not fetch or transform data beyond building its graph, and does not attach global event listeners to the page. It never decides *when* to move; it only resolves *where* a validated navigation command leads and renders what it is asked to render. This restraint is the practical meaning of the "technical material" framing discussed previously at the end of the chapter: like concrete, the library has no opinion about the building.

Our case examples therefore each supply four kinds of application code, and the division of labor is summarized in Figure 5.14. First, the structure itself: in our first and third case examples, every node, edge, and rule was authored by hand, while our second generated them from a *Vega-Lite* view. Second, modality adapters: the keyboard listener, the swipe recognizer, the hand gesture model, the speech recognition grammar, and the text command parser in our first case example are all demo code, and each one reduces an input event to the name of a navigation rule before handing control to the library. (This means that the application level does most of the hard work, parsing a gesture's swipe into the text string, "left" or equivalent.) Third, the focus lifecycle: when the input module returns the next node, the application renders it, focuses it, and removes the previously focused element. Fourth, coupling to the host visualization, such as highlighting the chart region that corresponds to the focused node.

A reader asking "what exactly do I get if I `npm install data-navigator`?" gets the graph builders, the traversal engine, the rendering layer, and the utilities; everything else in our demonstrations is replaceable application code, which is precisely the point.

The package is developed in a public monorepo that has continued to grow since the original publication of this work. At the time of writing it also hosts companion packages built on the core library, including `@data-navigator/inspector`, a visual graph debugger for navigation structures, and the *Skeleton* authoring application. We return to these in the following chapter, since this chapter concerns the core library.

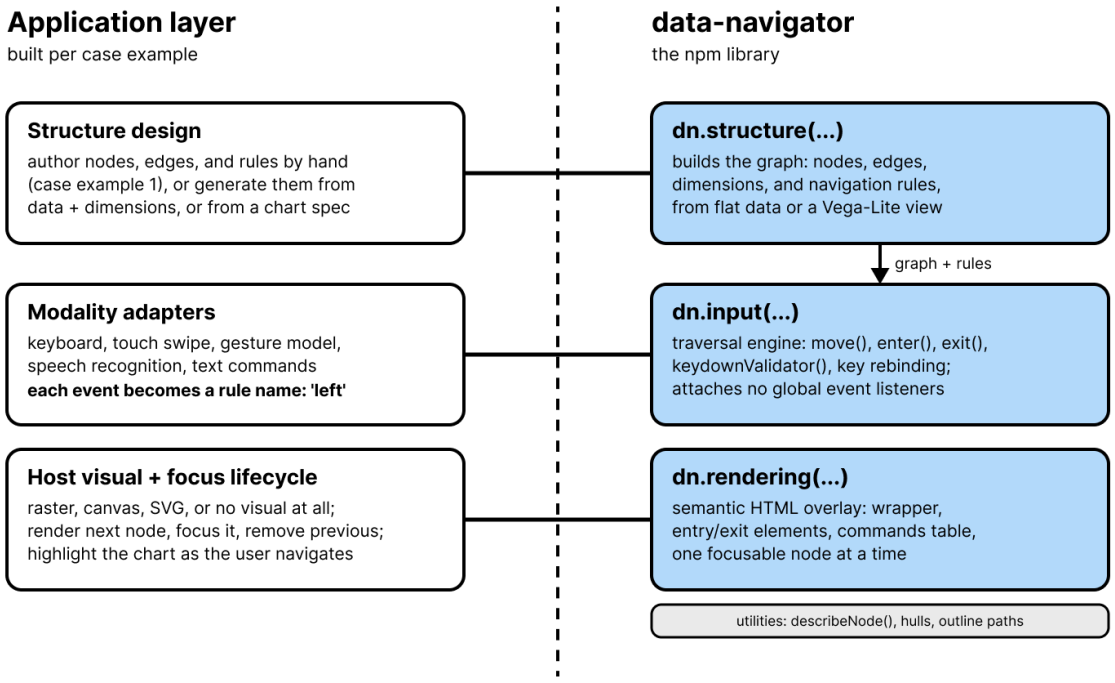


Figure 5.14: The library boundary in Data Navigator. The npm package (right) provides the graph builders, the traversal engine, the rendering layer, and supporting utilities. Each case example (left) supplies the structure design, the modality adapters that translate input events into navigation rule names, the focus lifecycle, and any coupling to the host visualization.

5.8.1.1 The structure module in implementation

In the implementation, a structure is a plain object containing `nodes`, `edges`, and `navigationRules`, plus optional `dimensions` and `elementData` (rendering properties keyed separately from the graph, so that structure and rendering remain independently specifiable). Nodes and edges are dictionaries keyed by identifier. A node object holds its `id` and an ordered list of edge *identifiers*, not edge objects; because nodes reference edges by id, a single edge object can appear in many nodes' lists, which is what makes the generic edges described above cheap. An edge object holds `source`, `target`, and `navigationRules`, the list of rule names it participates

in, where either endpoint may be a function of the current datum and current focus. A navigation rule object holds its orientation (`source` or `target`) and, optionally, a keyboard key.

Developers can author these objects entirely by hand, as in our first case example. The module's builders exist to remove that tedium when the data allows it. Passing `dataType: 'vega-lite'` instead dispatches to the *Vega-Lite* ingestion path used in our second case example, which walks a *Vega-Lite* view's scenegraph and emits nodes, edges, spatial properties, and descriptions, subject to developer-supplied inclusion criteria and an optional describer function.

The declarative grammar for dimensional scaffolding, and an authoring interface built on top of it, was not actually part of this chapter's work despite being in the library at the time of writing this postface. These are unpacked in the following chapter for *Skeleton*.

5.8.1.2 The input module in implementation

The input module is deliberately the smallest of the three: roughly a hundred lines that implement traversal and nothing else. `dn.input()` takes a structure, its navigation rules, and an entry and exit point, and returns a handler exposing `enter()`, `exit()`, `move(current, direction)`, `moveTo(id)`, `keydownValidator(event)`, `focus(renderId)`, and `setNavigationKeyBindings()`. The core, `move()`, implements the traversal function described in the structure subsection: it walks the current node's edge list in order, finds the first edge whose rule list contains the requested direction name, resolves the edge's endpoints (invoking them if they are functions), and returns the node at the rule's orientation, provided it is not the node the user is already on.

Two design choices here carry the input-agnosticism argument made above. First, `move()` takes a rule *name*, a plain string like `'left'` or `'drill-in'`, so any code that can produce a string can drive navigation; the swipe, gesture, speech, and text adapters in our first case example each end in exactly such a call. Second, the keyboard itself is handled the same way as every other modality: `keydownValidator()` is a pure mapping from a keyboard event code to a rule name (by default the arrow keys map to the four directions, Enter to drill-in, Escape to drill-out, and comma and period to backward and forward), and the bindings can be replaced wholesale through `setNavigationKeyBindings()`, which is the hook for end-user remapping mentioned above. The library attaches no global event listeners of its own; the host application listens for events where it sees fit, validates them, and forwards a direction. The library decides where a command leads, never when one occurs.

5.8.1.3 The rendering module in implementation

The rendering module maintains the semantic HTML layer. `dn.rendering()` is initialized with a root element and returns a renderer whose `initialize()` builds the scaffolding around the structure: a wrapper with `role="application"`, an accessible name, and a managed `aria-activedescendant` that tracks the focused node; an optional entry button ("Enter navigation area") that gives keyboard and screen reader users a discoverable way in; an optional exit element announced as the end of the data structure; and an optional, automatically generated commands table that lists every available navigation command and its key, derived directly from the structure's rules so that documentation cannot drift from behavior.

The renderer's `render()` then creates exactly one focusable element for a given node: an outer element whose tag, role, and attributes come from the node's parent semantics, an inner element carrying the required accessible label, a `tabindex` for focusability, and absolute positioning from the node's spatial properties. If a node specifies a path, the renderer additionally draws an SVG outline, which is how the bespoke cross-stack focus indication in Figure 5.8 is produced; the geometry utilities shipped with the package (convex hulls, unions of rectangles, and bounding outlines) exist to compute such paths for derived group nodes. Every rendering property is dynamic in the same sense as edge endpoints: each may be a static value or a function of the element's render data and datum, resolved at render time against a developer-supplied set of defaults. The renderer also enforces one hard accessibility rule: if a node reaches `render()` without a label, the library logs an explicit accessibility error rather than silently producing an unnamed focusable element. The complementary `remove()` and `clearStructure()` methods complete the on-demand lifecycle described above; in implementation, the render-then-remove swap is a convention enacted by the host application's focus lifecycle, which keeps the library free of assumptions about when an application wants elements to persist (as in pre-rendering) or disappear.

5.8.1.4 Graph-theoretic foundations

Having introduced nodes, edges, and navigation rules informally, we can now state what a Data Navigator structure formally is, because several of the system's guarantees follow directly from this formulation. A structure is a labeled multigraph $G = (V, E, R)$: a set of nodes V, a set of edges E where each edge carries a source endpoint, a target endpoint, and a non-empty set of labels drawn from R, and a set of navigation rules R where each rule has an orientation, either *source* or *target*. Endpoints may be node identifiers or functions that resolve to node identifiers at traversal time. Each node holds an ordered list of the edges that involve it. Navigation is then a partial function $move : V \times R \rightharpoonup V$: at node v, given rule r, the system scans v's edge list in order, takes the first edge labeled with r whose resolved endpoint in r's orientation differs from v, and moves there. If no such edge exists, the command is simply inert at that node.

The first is *determinism under multiplicity*. Because G is a multigraph, nothing prevents two edges at the same node from sharing a rule label; the ordered edge list resolves this by acting as a priority order, so every navigation command has exactly one outcome at every node. This is what lets designers attach broadly shared generic edges (such as the undo edge described above) to every node without worrying that they will shadow, or be shadowed by, more specific edges in unpredictable ways: precedence is explicit in the list order.

The second is *symmetry, and therefore undo-ability*. By default a single edge object serves a *pair* of rules with opposite orientations: `left` moves toward an edge's source while `right` moves toward its target, and the dimensional scaffolding described below always assigns rules in such pairs (`sibling_sibling` pairs like left and right or up and down, and `parent_child` pairs like drill-in and drill-out). The consequence is that every traversal step has an inverse along the same edge: a user who presses a key can press its partner and return to exactly where they were. Paths through the structure are reversible by construction, which means users can explore without memorizing routes, an essential property for non-visual navigation where there is no persistent overview to glance back at. The library also allows this symmetry to be deliberately

broken: the `useDirectedEdges` option reorients every rule toward its edge's target, producing one-way movement for genuinely directed structures such as flows and processes. Even then, reversibility can be restored globally by the generic return edge, a single shared edge whose endpoints resolve dynamically to the current and previous nodes, giving users an undo across any jump, directed or not.

The third is *reachability as a design property*. A structure is only as usable as the set of nodes reachable from its entry point, and the exit must be reachable from everywhere. The latter is cheap to guarantee in this formulation: one shared generic exit edge attached to every node makes the exit reachable in a single step from anywhere, which is how our case examples implement it. Reachability of the rest of the graph, however, is not statically guaranteed, precisely because endpoints may be functions: the realized graph is partially lazy, materialized edge by edge as users navigate. Expressiveness and verifiability trade off here. A designer hand-authoring edges can strand a node with no incoming edges, and no compiler will object. This is a genuine cost of the approach, and it is part of what motivated the inspection tooling presented in the following chapter, which renders a structure's graph so that designers can see disconnected or one-way regions before users encounter them.

Cycles in this formulation deserve a brief note, because they are the clearest illustration of why a graph substrate is the right choice. In the rendered tree of the bordering-states example (Figure 5.3), the cyclical relationships between neighboring states had to be materialized as repeated subtrees, producing redundancy that grows with the data. In a graph, a cycle is just a set of edges. Later in our *Dimensions API* explanation for *Skeleton* (next chapter), Data Navigator treats cycles as a controlled design dimension: each dimension's boundary behavior is declared through its `extents`, which determine whether sibling traversal wraps in a cycle (`circular`), terminates at the ends (`terminal`), or bridges out of the current division and into a neighboring one, skipping empty divisions and wrapping if needed (`bridgedCousins`), or into an explicitly chosen node (`bridgedCustom`). Whether a user's repeated keypress orbits a category, stops at its edge, or carries them into the next group is a one-word declaration.

Finally, the multi-dimensional structures shown earlier (Figure 5.9) have a precise reading in this formulation. Each declared dimension contributes a small rooted hierarchy over the *same* underlying data nodes: an optional shared root (level 0), the dimension's own node (level 1), its divisions (level 2), and the data nodes themselves (level 3). One tree over a shared leaf set is just a tree; the union of k of them is a graph in which each leaf has k parents, which is exactly the "two intersecting trees" of our first case example. Because every dimension is assigned its own pair of sibling rules, movement decomposes into independent axes: one pair of keys traverses siblings within one dimension while another pair traverses siblings within a second, all from the same node. The cost model is also worth stating: a move inspects only the current node's edge list, so each navigation step is proportional to that node's degree and the rules per edge, independent of $|V|$; storage is linear in nodes plus edges, with rule objects and generic edges shared rather than duplicated. The graph does not merely represent more structures than a tree, as argued above; it represents the same multi-dimensional structures more compactly and traverses them in constant local work.

Part VI

Conclusion

Chapter 9
Findings

The five projects in this thesis were presented as five chapters, each with its own results and its own conclusion. Read separately, they contribute a heuristic resource for evaluation, a structural toolkit for navigation, an authoring environment for non-visual design, an interaction technique and device for blind analysis, and a system and study of end-user personalization. This chapter states what they show *together*. Chapter 2 closed on a lopsided field: a breadth of empirical work and exemplary prototypes on one side, and a near absence of practitioner tools in production environments and practitioner action on the other, with the hardest remaining challenges concentrated in navigation, interaction, and personalization. The findings below are what this thesis learned by working inside that gap.

Each finding is a synthesis across chapters, and each is traceable to results reported in the body of this thesis. We have written them for two audiences at once: practitioners who build, evaluate, and maintain data experiences, and researchers who study how that work happens. Where a promising idea outran the evidence, it is not here; it is in the next chapter, with the rest of the discussion and future work.

9.2 Tools Can Reduce the Work Required to Make Things Accessible

Chapter 2 argued that guidance cannot substitute for means. The constructive half of that argument is that supplying means measurably changes what practitioners can do, and three chapters provide the evidence.

Chartability reduced the perceived difficulty of evaluation and increased the confidence of practitioners new to accessibility, whose audit results were reasonably comparable to the authors' own; beyond the study, it has been applied by policy and governmental organizations and more than 100 companies, across toolchains from Tableau and Excel to D3 and Figma. *Data Navigator* gave previously inaccessible rendering formats a navigable structure: a static raster image gained full multi-input navigation with no automatically detectable conformance failures, and a deterministic ingestion pipeline recreated, and then improved on, an existing toolkit's navigation scheme for rendering modes that previously had none. *Skeleton*'s scaffolding collapsed the spatial authoring of a navigation structure to one or two minutes of refinement, and within single sessions every participant iterated substantively on designs that, in a code-only workflow, our co-design collaborators had struggled to iterate on at all.

None of this is evidence that tools solve access; the boundaries of that claim get their own finding below. It is evidence of something narrower and more useful: the labor of access work is sensitive to tooling, and reductions in that labor are achievable, measurable, and repeatable across very different kinds of tools, from a workbook of heuristics to a rendering-independent toolkit to an authoring interface.

9.3 Making the Invisible Structurally Visible Changes Practice

The strongest through-line in this thesis is what happened when an invisible structure underlying practitioners' work was given a perceivable, inspectable form. Three chapters did this to three different structures. *Chartability* made the evaluative space visible: a scattered landscape of standards and research became a single workbook a practitioner could hold in view, and participants described it as helping them focus, stay consistent between sessions, and aim above compliance. *Skeleton* made navigation structure visible: nodes, edges, and announcements rendered over the practitioner's own chart, and its study found that participants who began by asking compliance questions (does this pass? what does the guideline say?) shifted to asking design questions (is this good? is this too many steps? what is this *for*?). *Softerware* made the dimensions of preference visible: an explicit menu of option categories gave end users something concrete to react to and gave practitioners a representation through which access friction stopped being abstract.

The mechanism is one visualization researchers already trust: reflective practice runs on externalized representations that a designer can perceive and respond to [114]. What this thesis adds is evidence that the loop works on structures the field had left invisible, and that restoring the loop moves practitioners from verification toward judgment. As a guideline, this is directly actionable for toolmakers: when practitioners treat some aspect of accessibility as a checkbox, ask what representation of that aspect they can actually perceive, because a structure no one can see is a structure no one can design.

Chapter 10
Discussion & Future Work

10.1 What is a "tool?" A reflection on the social and material identity of tools

In the introduction of this dissertation, we use the example of a hammer: a hammer can destroy and it can construct. So is the *use* of a technology what constitutes it? Do we understand the hammer as the *thing we swing, to destroy and to build*? Should we?

This thesis engages domains of tools and tool-making for accessibility: evaluation, navigation, interaction, and personalization. But these categories for work do not fully characterize the upstream conditions that our software systems and data interfaces inherit.

In our work specifically on accessibility, a larger social reality becomes apparent that shapes the question, "what is a tool?" far more than how an individual might use one, or the domains of work that our tool-making inhabits. Our research has navigated multiple social and political thresholds, from changes to law in the European Union, to the enactment of Title II as part of the update to the Americans with Disabilities Act. These laws have motivated a significant interest in accessibility research, solutions, guidelines, and technologies. In the midst of this, we have seen the rise of overlays and generative AI solutionism [53] and subsequent lawsuits and grass-roots resistance.

For our work, this is mostly good news. Legal change produces motivation, and even with predatory technology attempting to address real problems, pushback is widespread and active. But this paints a picture of the reality that our work inherits: many tools cannot even be used, or cease to be used, if there is not a social, political, and material set of conditions in place motivating those tools, providing resources for their construction, regulating their use, and examining the outcomes of what they accomplish. Tools and technologies are often a response to social, cultural, political, and legal realities that we first negotiate.

I, Frank, recently spoke on this at a keynote in Australia, on how a hammer isn't *just* a tool and that the idea that "the only thing that matters is how a tool is used" limits how we really understand tools. Instead, I spoke about how a standard, household hammer requires iron and wood. That alone leads to a whole universe of different questions. Western Australia's conservation efforts were disrupted when a significant amount of natural iron was discovered in a wildlife preserve. So laws were passed and now iron is mined there. That iron is largely exported. And Australia then, whether with Australian iron or not, mostly imports their small tools. Iron is sent out, and through a complex network of trade (likely indirectly related to the iron), hammers are brought in. A "hammer," to even exist at all, relies on multiple levels of human governance, international relations, and complex infrastructures of trade.

And while my metaphor is largely motivated to encourage younger practitioners to consider the "iron mines" in the technologies they use, such as modern generative AI, it is also an area that is not adequately explored and addressed in terms of accessibility research.

Research on accessibility is dependent on funding, which is often dependent on political

priorities and action. Depending on the current social and political state of the world at large, accessibility research itself may never gain the opportunities required in order to innovate and produce new tools at all. And as the US's 2025 federal cuts to research demonstrated, millions of dollars devoted to accessibility research can be lost to political agendas. It is for this reason that engagement with policy recommendation and guidance is essential. Personal political activity and involvement is also essential. Researchers who genuinely believe in accessibility as a human right or as a dignity that all people deserve should work with policymakers to ensure that there are material and structural resources in place for this work to continue. We cannot naively believe that technology, divorced from the realm of social and political forces, is capable of solving accessibility barriers [124]. Without enforcement and threat of litigation, very little accessibility work has been accomplished in the past by technology companies alone.

Featured in these chapters (in their pre and postfaces) is the policy and outreach work involved in seeing that work like *Chartability* and *Data Navigator* were used in real contexts, including by organizations that govern and influence the lives of many people. Immediate incentives to produce novelty may not be enough to sustain the larger socio-cultural and political ecosystems that our work participates in and is downstream from. We must also get involved.

10.3 Who is responsible for repair?

Lastly, we want to revisit one of this thesis's opening points, where we argue that the *tool-makers* are first responsible for repair. This is true. However, the most pressing issue we have faced in recent years is mostly unmentioned across these research projects: tool-makers might be responsible, but this is because they are the only ones who have the *power* to make things accessible. Does this always need to be the case? Can we imagine an artifact's authority over the user's ability to access being designed towards self-subversion [43] or de-centralized agency [19, 75, 98], instead? What might that look like, concretely?

In *Softerware*, we begin to engage this larger problem in terms of an idealized state where a user can repair or re-design their own experiences. But to me, this self-repair is like laying down train tracks for yourself as you move a locomotive, but then lifting up your own tracks behind you as you go. You're the only one helping yourself. This is not ideal, for you or others.

What we need are broad, lasting, infrastructural changes. On the web, this problem becomes quite difficult to solve. A personal computer or device? Again, someone can auto-design their interfaces into a better state. But back when we started *Chartability*, the WebAIM Million's report showed more than 95% of the top one million website home pages contain at least one critical accessibility error. And now, more than six years later, that proportion is unchanged [144].

Some had imagined that generative AI would solve the massive infrastructural repair problems we face. But unfortunately, the latest WebAIM Million report shows that since 2020, ARIA usage has increased and correlates to more errors, while use of tabindex on a page has increased nearly 300% and also correlates to more errors on a page. If anything, during the age of generative AI, we have seen existing bad patterns worsen in prevalence and complexity.

We firmly believe that a tools-based approach is not enough on its own. Tool-making cannot be the *only* intervention on inaccessibility. Tools and tool-making, as our thesis argues, have a powerful role to play. But we simply can't tool our way out of failed infrastructure and inadequate

policy when someone else *owns* the tools and tool-making. Visiting a website is like going into someone else's home: arranged according to their effort, tastes, and so on. If you can't access their home, you essentially need to request that they let you in personally. Website repair always falls to the owner and maintainer of a website, and they largely don't take any meaningful action.

Sidewalks outside of homes are a good parallel to this problem. Sidewalk accessibility is a massive infrastructural problem [106], and yet localities treat sidewalk maintenance in different ways: some, like where I, Frank, presently live in the south hills of Pittsburgh, put the onus on the homeowner whose house and property the sidewalk touches. In other places, sidewalks are considered a public path, similar to a roadway, and are maintained through public tax and resource management. To no surprise, privately-maintained, public-access sidewalks are worse for people in pretty much every way than publicly-maintained ones [151]. This is because private homeowners don't care about sidewalk maintenance unless the city manages to fine them or they get sued.

And the web is a collection of private spaces that you visit privately. There is no truly shared, universally democratic, public space on the web. Centralization is partly to blame: sharing space while scaling leads to consolidation.

So our future work will continue to wrestle with the same tensions of scale, repair, and anti-consolidation of power, motivated by the same WebAIM Million report. But now we look to questions of *democratic* and *radical* access to accessibility repair. The barriers we hope to tackle in the future are political and infrastructural. Perhaps tool-making will participate in this work, but it seems clear now from our work that the upstream technical problems and socio-political conditions that tools inherit, will likely not be addressed by tools alone.

What does "democratic" and "radical" infrastructure work look like? It will probably be an extension of *Softerware*, to some degree. We imagine future research that explores public-first spaces, ones where access is socially negotiated and repair belongs to all of us. Is this an autonomous space, like an autonomous zone [11] separate from the web? Above it, looking down into it, like shared annotation tools but capable of sharing the manipulation of websites [105]? A space with ambient co-repair, modeled after projects that bring people together [115] or that allow community "fixing" of misinformation [68]? Perhaps feminist thought on the ethics of care can help us [55, 88]? Or maybe it will be something else entirely; I'm not yet sure. But what made the web fantastic years ago is long gone; most of it has been hedged into corporate spaces that are controlled, maintained, and repaired by corporate power. And these entities are notoriously bad at repair. What we imagine in the future involves reclaiming a sense that the web is *ours*, belongs to *us*, and that ultimately *we* are responsible for making it accessible.

References

[1] Sara Ahmed. *Queer Phenomenology: Orientations, Objects, Others*. Duke University Press, Durham, NC, 2006. ISBN 9780822339144. ISBN: 978-0-8223-3914-4. 2.1.2

[2] Sara Ahmed. *What's the Use? On the Uses of Use*. Duke University Press, Durham, NC, 2019. ISBN 9781478006503. ISBN: 978-1478006503. 2.1.2

[3] R. Amar, J. Eagan, and J. Stasko. Low-level components of analytic activity in information visualization. In *IEEE Symposium on Information Visualization, 2005. INFOVIS 2005.*, page 111–117. IEEE, November 2005. doi: 10.1109/infvis.2005.1532136. URL https://doi.org/10.1109/infvis.2005.1532136. 3

[4] Apple Inc. Audio graphs. https://developer.apple.com/documentation/accessibility/audio-graphs, 2021. Apple Developer Documentation. Feature introduced at WWDC 2021. Accessed: 2026-06-12. 2.2.1

[5] Catherine M. Baker, Lauren R. Milne, Ryan Drapeau, Jeffrey Scofield, Cynthia L. Bennett, and Richard E. Ladner. Tactile graphics with a voice. *ACM Transactions on Accessible Computing*, 8(1):1–22, January 2016. ISSN 1936-7236. doi: 10.1145/2854005. URL https://doi.org/10.1145/2854005. 2.2.1

[6] BANA. Guidelines and Standards for Tactile Graphics. Braille standard, Braille Authority of North America, 2010. Accessed: 2021-09-03. 2.2.1

[7] Michel Beaudouin-Lafon, Susanne Bødker, and Wendy E. Mackay. Generative theories of interaction. *ACM Transactions on Computer-Human Interaction*, 28(6):1–54, November 2021. ISSN 1557-7325. doi: 10.1145/3468505. URL https://doi.org/10.1145/3468505. 2.1.2

[8] Cynthia L. Bennett, Erin Brady, and Stacy M. Branham. Interdependence as a Frame for Assistive Technology Research and Design. In *Proceedings of the 20th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '18, page 161–173, New York, NY, USA, October 2018. Association for Computing Machinery, ACM. doi: 10.1145/3234695.3236348. URL https://doi.org/10.1145/3234695.3236348. Accessed: 2021-09-03. 1.1, 2.3

[9] Cynthia L. Bennett, Burren Peil, and Daniela K. Rosner. Biographical prototypes. In *Proceedings of the 2019 on Designing Interactive Systems Conference*, pages 35–47. ACM, 2019. doi: 10.1145/3322276.3322376. 2.3

[10] Dan Bennett, Alan Dix, Parisa Eslambolchilar, Feng Feng, Tom Froese, Vassilis Kostakos, Sebastien Lerique, and Niels van Berkel. Emergent interaction: Complexity, dynamics,

and enaction in hci. In *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, CHI '21, page 1–7. ACM, May 2021. doi: 10.1145/3411763.3441321. URL https://doi.org/10.1145/3411763.3441321. 2.1.2

[11] Hakim Bey. *TAZ: The temporary autonomous zone, ontological anarchy, poetic terrorism*. Autonomedia, 1991. 10.3

[12] Brandon Biggs, Christopher Toth, Tony Stockman, James M. Coughlan, and Bruce N. Walker. Evaluation of a non-visual auditory choropleth and travel map viewer. In *Proceedings of the 27th International Conference on Auditory Display (ICAD 2022)*, pages 82–90. International Community for Auditory Display, June 2022. doi: 10.21785/icad2022.027. URL https://doi.org/10.21785/icad2022.027. PMCID: PMC10010675. 2.2.1

[13] Matt Blanco, Jonathan Zong, and Arvind Satyanarayan. Olli: An Extensible Visualization Library for Screen Reader Accessibility. In *IEEE VIS Posters*, 2022. URL https://vis.csail.mit.edu/pubs/olli. Accessed: 2026-03-30, https://vis.csail.mit.edu/pubs/olli. 2.2.1, 2.4.1, 5.4.3.1, 5.4.3.2, 5.4.3.3

[14] M. Bostock, V. Ogievetsky, and J. Heer. D³ data-driven documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, December 2011. ISSN 1077-2626. doi: 10.1109/tvcg.2011.185. URL https://doi.org/10.1109/tvcg.2011.185. 1.2, 2.2.3, 5.5.1.1

[15] Geoffrey C. Bowker and Susan Leigh Star. *Sorting Things Out: Classification and Its Consequences*. MIT Press, Cambridge, MA, 1999. ISBN 9780262522953. ISBN: 978-0-262-52295-3. 2.1.2

[16] L.H. Boyd, W.L. Boyd, and G.C. Vanderheiden. The graphical user interface: Crisis, danger, and opportunity. *Journal of Visual Impairment & Blindness*, 84(10):496–502, December 1990. ISSN 1559-1476. doi: 10.1177/0145482x9008401002. URL https://doi.org/10.1177/0145482x9008401002. 5.3.2.3

[17] S. Brewster. Visualization tools for blind people using multiple modalities. *Disability and Rehabilitation*, 24(11-12):613–621, January 2002. ISSN 1464-5165. doi: 10.1080/09638280110111388. URL https://doi.org/10.1080/09638280110111388. 2.2.1

[18] Jennifer L. Burke, Matthew S. Prewett, Ashley A. Gray, Liuquin Yang, Frederick R. B. Stilson, Michael D. Coovert, Linda R. Elliot, and Elizabeth Redden. Comparing the effects of visual-auditory and visual-tactile feedback on user performance: a meta-analysis. In *Proceedings of the 8th international conference on Multimodal interfaces*, ICMI06, page 108–117. ACM, November 2006. doi: 10.1145/1180995.1181017. URL https://doi.org/10.1145/1180995.1181017. 1.1

[19] Katherine Casey. Radical decentralization: Does community-driven development work? *Annual Review of Economics*, 10(1):139–163, August 2018. ISSN 1941-1391. doi: 10.1146/annurev-economics-080217-053339. URL http://dx.doi.org/10.1146/annurev-economics-080217-053339. 10.3

[20] James I. Charlton. *Nothing About Us Without Us: Disability Oppression and Empower-*

ment. University of California Press, Berkeley, CA, March 1998. ISBN 9780520925441. doi: 10.1525/9780520925441. URL https://doi.org/10.1525/9780520925441. 2.3

[21] Pramod Chundury, Biswaksen Patnaik, Yasmin Reyazuddin, Christine Tang, Jonathan Lazar, and Niklas Elmqvist. Towards understanding sensory substitution for accessible visualization: An interview study. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):1084–1094, January 2022. ISSN 2160-9306. doi: 10.1109/tvcg.2021.3114829. URL https://doi.org/10.1109/tvcg.2021.3114829. 2.2.1, 5.3.1.2

[22] Pramod Chundury, Urja Thakkar, Yasmin Reyazuddin, J. Bern Jordan, Niklas Elmqvist, and Jonathan Lazar. Understanding the visualization and analytics needs of blind and low-vision professionals. In *The 26th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '24, page 1–5. ACM, October 2024. doi: 10.1145/3663548.3688496. URL https://doi.org/10.1145/3663548.3688496. 2.1.1

[23] Clare Cullen and Oussama Metatla. Co-designing inclusive multisensory story mapping with children with mixed visual abilities. In *Proceedings of the 18th ACM International Conference on Interaction Design and Children*, IDC '19, pages 361–373. ACM, June 2019. doi: 10.1145/3311927.3323146. URL https://doi.org/10.1145/3311927.3323146. 2.2.1

[24] Lilian de Greef, Dominik Moritz, and Cynthia Bennett. Interdependent variables: Remotely designing tactile graphics for an accessible workflow. In *Proceedings of the 23rd International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '21, page 1–6. ACM, October 2021. doi: 10.1145/3441852.3476468. URL https://doi.org/10.1145/3441852.3476468. 2.2.1, 5.5.3.1

[25] Deque. Automated Testing Identifies 57 Percent of Digital Accessibility Issues. Technical report, Deque, March 2021. Accessed: 2021-09-06. 3

[26] Alan Dix. Designing for appropriation. In *Electronic Workshops in Computing*. BCS Learning & Development, September 2007. doi: 10.14236/ewic/hci2007.53. URL https://doi.org/10.14236/ewic/hci2007.53. 2.1.2

[27] Dot Pad inc. Dot pad - the first tactile graphics display for the visually impaired. https://pad.dotincorp.com/, 2020. [Accessed 01-04-2025]. 5.5.3.1

[28] Paul Dourish. The appropriation of interactive technologies: Some lessons from placeless documents. *Computer Supported Cooperative Work (CSCW)*, 12(4):465–490, December 2003. ISSN 1573-7551. doi: 10.1023/a:1026149119426. URL https://doi.org/10.1023/a:1026149119426. 5.6

[29] Sebastian Draxler, Gunnar Stevens, Martin Stein, Alexander Boden, and David Randall. Supporting the social context of technology appropriation: on a synthesis of sharing tools and tool knowledge. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '12, page 2835–2844. ACM, May 2012. doi: 10.1145/2207676.2208687. URL https://doi.org/10.1145/2207676.2208687. 2.1.2

[30] Eloi Durant, Mathieu Rouard, Eric W. Ganko, Cedric Muller, Alan M. Cleary, Andrew D.

Farmer, Matthieu Conte, and Francois Sabot. Ten simple rules for developing visualization tools in genomics. *PLOS Computational Biology*, 18(11):e1010622, November 2022. ISSN 1553-7358. doi: 10.1371/journal.pcbi.1010622. URL https://doi.org/10.1371/journal.pcbi.1010622. 5.3.1.3

[31] Frank Elavsky. Method and system for accessible data visualization on a web platform. *Defensive Publication Series*, 4220, April 2021. 5.4.3.3

[32] Frank Elavsky, Cynthia Bennett, and Dominik Moritz. How accessible is my visualization? evaluating visualization accessibility with chartability. *Computer Graphics Forum*, 41(3):57–70, June 2022. doi: 10.1111/cgf.14522. URL https://doi.org/10.1111/cgf.14522. 5.3.1, 5.3.1.3, 5.5.1.1, 5.6

[33] Danyang Fan, Alexa Fay Siu, Hrishikesh Rao, Gene Sung-Ho Kim, Xavier Vazquez, Lucy Greco, Sile O'Modhrain, and Sean Follmer. The accessibility of data visualizations on the web for screen reader users: Practices and experiences during covid-19. *ACM Transactions on Accessible Computing*, 16(1):1–29, March 2023. ISSN 1936-7236. doi: 10.1145/3557899. URL https://doi.org/10.1145/3557899. 1.1, 5.3.1, 5.3.1.2, 5.3.1.3

[34] Stephen Few. Visual Business Intelligence - Data Visualization and the Blind. https://www.perceptualedge.com/blog/?p=1756, 2013. [Accessed 05-04-2025]. 1.1

[35] Chris Fleizach and Jeffrey P. Bigham. System-class accessibility: The architectural support for making a whole system usable by people with disabilities. *Queue*, 22(5):28–39, October 2024. ISSN 1542-7749. doi: 10.1145/3704627. URL http://dx.doi.org/10.1145/3704627. 2.2.2

[36] John H. Flowers, Dion C. Buhman, and Kimberly D. Turnage. Cross-Modal Equivalence of Visual and Auditory Scatterplots for Exploring Bivariate Data Samples. *Human Factors: The Journal of the Human Factors and Ergonomics Society*, 39(3):341–351, September 1997. doi: 10.1518/001872097778827151. Accessed: 2021-09-03. 2.2.1, 3

[37] Steven L. Franconeri, Lace M. Padilla, Priti Shah, Jeffrey M. Zacks, and Jessica Hullman. The science of visual data communication: What works. *Psychological Science in the Public Interest*, 22(3):110–161, December 2021. ISSN 1539-6053. doi: 10.1177/15291006211051956. URL https://doi.org/10.1177/15291006211051956. 1.2

[38] Krzysztof Z. Gajos, Daniel S. Weld, and Jacob O. Wobbrock. Automatically generating personalized user interfaces with SUPPLE. *Artificial Intelligence*, 174(12–13):910–950, August 2010. ISSN 0004-3702. doi: 10.1016/j.artint.2010.05.005. URL https://doi.org/10.1016/j.artint.2010.05.005. 2.4.3

[39] Emden R. Gansner and Stephen C. North. An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233, September 2000. ISSN 1097-024X. doi: 10.1002/1097-024x(200009)30:11⟨1203::aid-spe338⟩3.0.co;2-n. URL https://doi.org/10.1002/1097-024x(200009)30:11<1203::aid-spe338>3.0.co;2-n. 5.4.1.1

[40] Stéphanie Giraud and Christophe Jouffrais. Empowering low-vision rehabilitation profes-

sionals with "do-it-yourself" methods. In *Lecture Notes in Computer Science*, page 61–68. Springer International Publishing, 2016. ISBN 9783319412672. doi: 10.1007/978-3-319-41267-2_9. URL https://doi.org/10.1007/978-3-319-41267-2_9. 5.3.3

[41] Michele E. Gloede and Melissa K. Gregg. The fidelity of visual and auditory memory. *Psychonomic Bulletin & Review*, 26(4):1325–1332, April 2019. ISSN 1531-5320. doi: 10.3758/s13423-019-01597-7. URL https://doi.org/10.3758/s13423-019-01597-7. 1.1

[42] A. Jonathan R. Godfrey, Paul Murrell, and Volker Sorge. An accessible interaction model for data visualisation in statistics. In *Lecture Notes in Computer Science*, page 590–597. Springer International Publishing, 2018. ISBN 9783319942773. doi: 10.1007/978-3-319-94277-3_92. URL https://doi.org/10.1007/978-3-319-94277-3_92. 2.2.1, 5.2, 5.3.1.1, 5.3.2.1

[43] David Graeber and Charlie Rose. Never cower to authority, 2006. URL https://www.youtube.com/watch?v=qt5op6wft-8. Accessed [2026]. 10.3

[44] David Gray Widder, Sarah West, and Meredith Whittaker. Open (for business): Big tech, concentrated power, and the political economy of open ai. *SSRN Electronic Journal*, 2023. ISSN 1556-5068. doi: 10.2139/ssrn.4543807. URL https://doi.org/10.2139/ssrn.4543807. 1.2

[45] Catherine Grevet and Eric Gilbert. Piggyback prototyping: Using existing, large-scale social computing systems to prototype new ones. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*, CHI '15, page 4047–4056. ACM, April 2015. doi: 10.1145/2702123.2702395. URL https://doi.org/10.1145/2702123.2702395. 2.1.2

[46] Nathan Hahn, Joseph Chang, Ji Eun Kim, and Aniket Kittur. The knowledge accelerator: Big picture thinking in small pieces. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, CHI'16, page 2258–2270. ACM, May 2016. doi: 10.1145/2858036.2858364. URL https://doi.org/10.1145/2858036.2858364. 2.1.1

[47] Jen Hale. Extensive digitization of tactile map collection. https://www.perkins.org/extensive-digitization-of-tactile-map-collection/, 2016. Perkins Archives Blog, Perkins School for the Blind. July 2016. Accessed: 2022-02-23. 2.2.1

[48] Foad Hamidi, Patrick Mbullo, Deurence Onyango, Michaela Hynie, Susan McGrath, and Melanie Baljko. Participatory design of diy digital assistive technology in western kenya. In *Proceedings of the Second African Conference for Human Computer Interaction: Thriving Communities*, AfriCHI '18, page 1–11. ACM, December 2018. doi: 10.1145/3283458.3283478. URL https://doi.org/10.1145/3283458.3283478. 2.1.1

[49] A. Hamraie. Making access critical: Disability, race, and gender in environmental design. https://belonging.berkeley.edu/aimi-hamraie-making-access-critical-disability-race-and-gender-environmental-design/,

2019. [Online]. 2.3, 3

[50] Aimi Hamraie. *Building Access: Universal Design and the Politics of Disability*. University of Minnesota Press, Minneapolis, MN, 2017. ISBN 9781517901639. ISBN: 978-1-5179-0163-9. 2.3, 2.4.3

[51] Aimi Hamraie. Designing collective access: A feminist disability theory of universal design. In *Disability, space, architecture*, pages 78–297. Routledge, 2017. 2.3

[52] Aimi Hamraie and Kelly Fritsch. Crip technoscience manifesto. *Catalyst: Feminism, Theory, Technoscience*, 5(1):1–33, April 2019. ISSN 2380-3312. doi: 10.28968/cftt.v5i1.29607. URL https://doi.org/10.28968/cftt.v5i1.29607. 2.3

[53] Parker Hartman and Tim Gorichanaz. Evaluating ai-powered website accessibility overlays. In *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '25, page 1–4. ACM, October 2025. doi: 10.1145/3663547.3759761. URL http://dx.doi.org/10.1145/3663547.3759761. 10.1

[54] Jeffrey Heer and Dominik Moritz. Mosaic: An architecture for scalable & interoperable data views. *IEEE Transactions on Visualization and Computer Graphics*, page 1–11, 2023. ISSN 2160-9306. doi: 10.1109/tvcg.2023.3327189. URL https://doi.org/10.1109/tvcg.2023.3327189. 1.2, 2.2.3, 2.4.2, 3

[55] Ana O Henriques, Anna R. L. Carter, Beatriz Severes, Reem Talhouk, Angelika Strohmayer, Ana Cristina Pires, Colin M. Gray, Kyle Montague, and Hugo Nicolau. A feminist care ethics toolkit for community-based design: Bridging theory and practice. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25, page 1–26. ACM, April 2025. doi: 10.1145/3706598.3713950. URL http://dx.doi.org/10.1145/3706598.3713950. 10.3

[56] Thomas Hermann, Andy Hunt, and John G Neuhoff, editors. *The Sonification Handbook*. Logos Verlag Berlin, Berlin, Germany, December 2011. ISBN 9783832528195. ISBN: 978-3-8325-2819-5. 2.2.1

[57] Jaylin Herskovitz, Andi Xu, Rahaf Alharbi, and Anhong Guo. Hacking, switching, combining: Understanding and supporting diy assistive technology design by blind people. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, page 1–17. ACM, April 2023. doi: 10.1145/3544548.3581249. URL https://doi.org/10.1145/3544548.3581249. 2.1.2

[58] Highsoft. Highcharts - interactive charting library for developers. https://www.highcharts.com/, 2009. [Online]. 2.2.1

[59] Highsoft. Stacked column demo. https://www.highcharts.com/demo/column-stacked, 2018. Accessed: 2022-12-31. 5.5.1

[60] Highsoft. Highcharts accessibility module: information and demos. https://highcharts.com/docs/accessibility/accessibility-module, 2018. Accessed: 2021-09-06. 5.3.2.1

[61] Highsoft. Highcharts: Render millions of chart points with the boost module. https://www.highcharts.com/blog/tutorials/highcharts-high-perform

ance-boost-module/, 2020. Accessed: 2022-11-20. 5.5.1

[62] Megan Hofmann, Devva Kasnitz, Jennifer Mankoff, and Cynthia L Bennett. Living disability theory: Reflections on access, research, and design. In *Proceedings of the 22nd International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '20, page 1–13. ACM, October 2020. doi: 10.1145/3373625.3416996. URL http://dx.doi.org/10.1145/3373625.3416996. 2.3

[63] Fred Hohman, Mary Beth Kery, Donghao Ren, and Dominik Moritz. Model compression in practice: Lessons learned from practitioners creating on-device machine learning experiences. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, CHI '24, page 1–18. ACM, May 2024. doi: 10.1145/3613904.3642109. URL http://dx.doi.org/10.1145/3613904.3642109. 2.1.2

[64] Jonathan Hook, Sanne Verbaan, Abigail Durrant, Patrick Olivier, and Peter Wright. A study of the challenges related to diy assistive technology in the context of children with disabilities. In *Proceedings of the 2014 conference on Designing interactive systems*, DIS '14, pages 597–606. ACM, June 2014. doi: 10.1145/2598510.2598530. URL https://doi.org/10.1145/2598510.2598530. 2.1.1

[65] Stacy Hsueh, Beatrice Vincenzi, Akshata Murdeshwar, and Marianela Ciolfi Felice. Cripping data visualizations: Crip technoscience as a critical lens for designing digital access. In *The 25th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '23, page 1–16. ACM, October 2023. doi: 10.1145/3597638.3608427. URL http://dx.doi.org/10.1145/3597638.3608427. 1.1, 2.3, 3

[66] Amy Hurst and Shaun Kane. Making "making" accessible. In *Proceedings of the 12th International Conference on Interaction Design and Children*, IDC '13, pages 635–638, New York, NY, USA, June 2013. Association for Computing Machinery, ACM. doi: 10.1145/2485760.2485883. Accessed: 2021-09-03. 2.1.1, 5.3.3

[67] Amy Hurst and Jasmine Tobias. Empowering individuals with do-it-yourself assistive technology. In *The proceedings of the 13th international ACM SIGACCESS conference on Computers and accessibility*, ASSETS '11, page 11–18. ACM, October 2011. doi: 10.1145/2049536.2049541. URL https://doi.org/10.1145/2049536.2049541. 2.1.1

[68] Farnaz Jahanbakhsh, Amy X. Zhang, Karrie Karahalios, and David R. Karger. Our browser extension lets readers change the headlines on news articles, and you won't believe what they did! *Proceedings of the ACM on Human-Computer Interaction*, 6 (CSCW2):1–33, November 2022. ISSN 2573-0142. doi: 10.1145/3555643. URL http://dx.doi.org/10.1145/3555643. 10.3

[69] Chutian Jiang, Wentao Lei, Emily Kuang, Teng Han, and Mingming Fan. Understanding strategies and challenges of conducting daily data analysis (dda) among blind and low-vision people. In *The 25th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '23, page 1–15. ACM, October 2023. doi: 10.1145/3597638.3608423. URL https://doi.org/10.1145/3597638.3608423. 2.1.1

[70] Shuli Jones, Isabella Pedraza Pineros, Daniel Hajas, Jonathan Zong, and Arvind Satya-

narayan. "customization is key": Reconfigurable textual tokens for accessible data visualizations. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, CHI '24, page 1–14. ACM, May 2024. doi: 10.1145/3613904.3641970. URL https://doi.org/10.1145/3613904.3641970. 2.4.3, 3

[71] Shakila Cherise S Joyner, Amalia Riegelhuth, Kathleen Garrity, Yea-Seul Kim, and Nam Wook Kim. Visualization accessibility in the wild: Challenges faced by visualization designers. In *CHI Conference on Human Factors in Computing Systems*, CHI '22, page 1–19. ACM, April 2022. doi: 10.1145/3491102.3517630. URL https://doi.org/10.1145/3491102.3517630. 2.2.3, 3

[72] Crescentia Jung, Shubham Mehta, Atharva Kulkarni, Yuhang Zhao, and Yea-Seul Kim. Communicating visualizations without visuals: Investigation of visualization alternative text for people with visual impairments. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):1095–1105, January 2022. ISSN 2160-9306. doi: 10.1109/tvcg.2021.3114846. URL https://doi.org/10.1109/tvcg.2021.3114846. 2.2.1, 2.4, 5.2, 5.3.1.2

[73] Shaun K. Kane, Jeffrey P. Bigham, and Jacob O. Wobbrock. Slide rule: Making mobile touch screens accessible to blind people using multi-touch interaction techniques. In *Proceedings of the 10th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS08, page 73–80. ACM, October 2008. doi: 10.1145/1414471.1414487. URL https://doi.org/10.1145/1414471.1414487. 2.4.2

[74] Hyeonsu B. Kang, Xin Qian, Tom Hope, Dafna Shahaf, Joel Chan, and Aniket Kittur. Augmenting scientific creativity with an analogical search engine. *ACM Transactions on Computer-Human Interaction*, 29(6):1–36, November 2022. ISSN 1557-7325. doi: 10.1145/3530013. URL https://doi.org/10.1145/3530013. 2.1.1

[75] Os Keyes, Josephine Hoy, and Margaret Drouhard. Human-computer insurrection: Notes on an anarchist hci. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, page 1–13. ACM, May 2019. doi: 10.1145/3290605.3300569. URL http://dx.doi.org/10.1145/3290605.3300569. 10.3

[76] Jiho Kim, Arjun Srinivasan, Nam Wook Kim, and Yea-Seul Kim. Exploring chart question answering for blind and low vision users. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, page 1–15. ACM, April 2023. doi: 10.1145/3544548.3581532. URL https://doi.org/10.1145/3544548.3581532. 2.2.1, 5.3.1.2

[77] N. W. Kim, S. C. Joyner, A. Riegelhuth, and Y. Kim. Accessible visualization: Design space, opportunities, and challenges. *Computer Graphics Forum*, 40(3):173–188, June 2021. doi: 10.1111/cgf.14298. URL https://doi.org/10.1111/cgf.14298. 2.2.3, 5.6

[78] N. W. Kim, G. Ataguba, S. C. Joyner, Chuangdian Zhao, and Hyejin Im. Beyond alternative text and tables: Comparative analysis of visualization tools and accessibility methods. *Computer Graphics Forum*, 42(3):323–335, June 2023. ISSN 1467-8659. doi: 10.1111/cgf.14833. URL https://doi.org/10.1111/cgf.14833. 2.4.3

[79] Nicolas Kruchten, Jon Mease, and Dominik Moritz. Vegafusion: Automatic server-side scaling for interactive vega visualizations. In *2022 IEEE Visualization and Visual Analytics (VIS)*, page 11–15. IEEE, October 2022. doi: 10.1109/vis54862.2022.00011. URL http://dx.doi.org/10.1109/VIS54862.2022.00011. 5.8

[80] David Ledo, Steven Houben, Jo Vermeulen, Nicolai Marquardt, Lora Oehlberg, and Saul Greenberg. Evaluation strategies for HCI toolkit research. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI '18, page 1–17. ACM, April 2018. doi: 10.1145/3173574.3173610. URL https://doi.org/10.1145/3173574.3173610. 2.1.2, 2.2.3

[81] Chien-Yu Lin and Yu-Ming Chang. Increase in physical activities in kindergarten children with cerebral palsy by employing makey–makey-based task systems. *Research in Developmental Disabilities*, 35(9):1963–1969, September 2014. doi: 10.1016/j.ridd.2014.04.028. URL https://doi.org/10.1016/j.ridd.2014.04.028. 5.3.3

[82] Zhicheng Liu and Jeffrey Heer. The effects of interactive latency on exploratory visual analysis. *IEEE Transactions on Visualization and Computer Graphics*, 20(12):2122–2131, December 2014. ISSN 1077-2626. doi: 10.1109/tvcg.2014.2346452. URL https://doi.org/10.1109/tvcg.2014.2346452. 2.4.2, 3

[83] Panagiotis Louridas. Design as bricolage: anthropology meets design thinking. *Design Studies*, 20(6):517–535, November 1999. ISSN 0142-694X. doi: 10.1016/s0142-694x(98)00044-1. URL https://doi.org/10.1016/s0142-694x(98)00044-1. 5.6

[84] Alan Lundgard and Arvind Satyanarayan. Accessible visualization via natural language descriptions: A four-level model of semantic content. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):1073–1083, January 2022. ISSN 2160-9306. doi: 10.1109/tvcg.2021.3114770. URL https://doi.org/10.1109/tvcg.2021.3114770. 2.2.1, 2.4, 5.3.1.2, 5.3.1.3, 5.5.1.1, 5.5.3, 5.6

[85] Alan Lundgard, Crystal Lee, and Arvind Satyanarayan. Sociotechnical considerations for accessible visualization design. In *2019 IEEE Visualization Conference (VIS)*, page 16–20. IEEE, October 2019. doi: 10.1109/visual.2019.8933762. URL https://doi.org/10.1109/visual.2019.8933762. 5.3.1.3, 5.5.1.1, 5.5.3, 5.6

[86] Dor Ma'ayan, Wode Ni, Katherine Ye, Chinmay Kulkarni, and Joshua Sunshine. How domain experts create conceptual diagrams and implications for tool design. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, CHI '20, page 1–14, New York, NY, USA, April 2020. ACM. ISBN 9781450367080. doi: 10.1145/3313831.3376253. URL https://doi.org/10.1145/3313831.3376253. 5.4, 5.6

[87] Kelly Mack, Emma McDonnell, Dhruv Jain, Lucy Lu Wang, Jon E. Froehlich, and Leah Findlater. What do we mean by "accessibility research"?: A literature survey of accessibility papers in chi and assets from 1994 to 2019. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, CHI '21, page 1–18, New York, NY, USA, May 2021. ACM. ISBN 978-1-4503-8096-6. doi: 10.1145/3411764.3445412. URL https://doi.org/10.1145/3411764.3445412. Accessed: 2022-02-22.

2.2.3

[88] Els Maeckelberghe. Feminist ethic of care: A third alternative approach. *Health Care Analysis*, 12(4):317–327, December 2004. ISSN 1573-3394. doi: 10.1007/s10728-004-6639-6. URL http://dx.doi.org/10.1007/s10728-004-6639-6. 10.3

[89] R G Maling and D C Clarkson. Electronic controls for the tetraplegic (possum) (patient operated selector mechanisms—p.o. s.m.). *Spinal Cord*, 1(3):161–174, November 1963. ISSN 1476-5624. doi: 10.1038/sc.1963.15. URL http://dx.doi.org/10.1038/sc.1963.15. 3

[90] Douglass L. Mansur, Merra M. Blattner, and Kenneth I. Joy. Sound graphs: A numerical data analysis method for the blind. *Journal of Medical Systems*, 9(3):163–174, June 1985. ISSN 1573-689X. doi: 10.1007/bf00996201. URL https://doi.org/10.1007/bf00996201. 2.2.1

[91] Deborah Marks. Models of disability. *Disability and Rehabilitation*, 19(3):85–91, January 1997. ISSN 1464-5165. doi: 10.3109/09638289709166831. URL https://doi.org/10.3109/09638289709166831. 1.1

[92] Kim Marriott, Bongshin Lee, Matthew Butler, Ed Cutrell, Kirsten Ellis, Cagatay Goncu, Marti Hearst, Kathleen McCoy, and Danielle Albers Szafir. Inclusive data visualization for people with disabilities: a call to action. *Interactions*, 28(3):47–51, April 2021. ISSN 1558-3449. doi: 10.1145/3457875. URL https://doi.org/10.1145/3457875. 2.2.3, 5.3.1, 5.3.3, 5.6

[93] David K. McGookin and Stephen A. Brewster. Soundbar: Exploiting multiple views in multimodal graph browsing. In *Proceedings of the 4th Nordic conference on Human-computer interaction: Changing roles*, NORDICHI06, page 145–154, New York, NY, USA, October 2006. Association for Computing Machinery, ACM. doi: 10.1145/1182475.1182491. URL https://doi.org/10.1145/1182475.1182491. Accessed: 2021-09-03. 2.2.1

[94] Andrew M. McNutt. No grammar to rule them all: A survey of json-style dsls for visualization. *IEEE Transactions on Visualization and Computer Graphics*, page 1–11, 2022. ISSN 2160-9306. doi: 10.1109/tvcg.2022.3209460. URL https://doi.org/10.1109/tvcg.2022.3209460. 2.1.2

[95] Andrew Michael McNutt. *Understanding and Enhancing JSON-based DSL Interfaces for Visualization*. PhD thesis, University of Chicago, 2023. URL https://doi.org/10.31219/osf.io/fy246. 2.1.2

[96] Catherine Mei∗, Josh Pollock∗, Daniel Hajas, Jonathan Zong, and Arvind Satyanarayan. Benthic: Perceptually congruent structures for accessible charts and diagrams. In *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '25, page 1–17. ACM, October 2025. doi: 10.1145/3663547.3746342. URL https://doi.org/10.1145/3663547.3746342. 2.2.1

[97] Mia Mingus. Access intimacy: The missing link. https://leavingevidence.wordpress.com/2011/05/05/access-intimacy-the-missing-link/, May 2011. Blog post, *Leaving Evidence*. 1.1, 2.3

[98] Giles Mohan and Kristian Stokke. Participatory development and empowerment: The dangers of localism. *Third World Quarterly*, 21(2):247–268, April 2000. ISSN 1360-2241. doi: 10.1080/01436590050004346. URL http://dx.doi.org/10.1080/01436590050004346. 10.3

[99] Dominik Moritz, Bill Howe, and Jeffrey Heer. Falcon: Balancing interactive latency and resolution sensitivity for scalable linked visualizations. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, page 1–11. ACM, May 2019. doi: 10.1145/3290605.3300924. URL https://doi.org/10.1145/3290605.3300924. 3

[100] Catherine A. Okoro, NaTasha D. Hollis, Alissa C. Cyrus, and Shannon Griffin-Blake. Prevalence of disabilities and health care access by disability status and type among adults — united states, 2016. *MMWR. Morbidity and Mortality Weekly Report*, 67(32):882–887, August 2018. ISSN 1545-861X. doi: 10.15585/mmwr.mm6732a3. URL https://doi.org/10.15585/mmwr.mm6732a3. 1.1

[101] Christopher Power, André Freire, Helen Petrie, and David Swallow. Guidelines are only half of the story: Accessibility problems encountered by blind users on the web. In *Proceedings of the SIGCHI conference on human factors in computing systems*, CHI '12, page 433–442, New York, NY, USA, May 2012. Association for Computing Machinery, ACM. doi: 10.1145/2207676.2207736. URL https://doi.org/10.1145/2207676.2207736. 2.2.2, 2.3, 3

[102] Blake Ellis Reid. The curb-cut effect and the perils of accessibility without disability. *SSRN Electronic Journal*, 2022. ISSN 1556-5068. doi: 10.2139/ssrn.4262991. URL https://doi.org/10.2139/ssrn.4262991. 1.1, 5.6

[103] Yvonne Rogers, Jeni Paay, Margot Brereton, Kate L. Vaisutis, Gary Marsden, and Frank Vetere. Never too old: engaging retired people inventing the future with makey makey. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '14, page 3913–3922. ACM, April 2014. doi: 10.1145/2556288.2557184. URL https://doi.org/10.1145/2556288.2557184. 5.3.3, 5.4.2.1

[104] Rod D Roscoe, Erin K Chiou, and Abigail R Wooldridge, editors. *Advancing diversity, inclusion, and social justice through human systems engineering*. CRC Press, London, England, December 2019. 1.1

[105] Heather Ruland Staines. Digital open annotation with hypothesis: Supplying the missing capability of the web. *Journal of Scholarly Publishing*, 49(3):345–365, April 2018. ISSN 1710-1166. doi: 10.3138/jsp.49.3.04. URL http://dx.doi.org/10.3138/jsp.49.3.04. 10.3

[106] Manaswi Saha, Michael Saugstad, Hanuma Teja Maddali, Aileen Zeng, Ryan Holland, Steven Bower, Aditya Dash, Sage Chen, Anthony Li, Kotaro Hara, and Jon Froehlich. Project sidewalk: A web-based crowdsourcing tool for collecting sidewalk accessibility data at scale. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, page 1–14. ACM, May 2019. doi: 10.1145/3290605.3300292. URL http://dx.doi.org/10.1145/3290605.3300292. 10.3

[107] Antti Salovaara. Inventing new uses for tools: A cognitive foundation for studies on appropriation. *Human Technology: An Interdisciplinary Journal on Humans in ICT Environments*, 4(2):209–228, November 2008. ISSN 1795-6889. doi: 10.17011/ht/urn.2008 11065856. URL https://doi.org/10.17011/ht/urn.200811065856. 2.1.2

[108] Elizabeth B.-N. Sanders and Pieter Jan Stappers. Probes, toolkits and prototypes: three approaches to making in codesigning. *CoDesign*, 10(1):5–14, January 2014. ISSN 1745-3755. doi: 10.1080/15710882.2014.888183. URL https://doi.org/10.1080/15710882.2014.888183. 2.1.1

[109] Zhanna Sarsenbayeva, Niels van Berkel, Eduardo Velloso, Jorge Goncalves, and Vassilis Kostakos. Methodological standards in accessibility research on motor impairments: A survey. *ACM Computing Surveys*, 55(7):1–35, December 2022. ISSN 1557-7341. doi: 10.1145/3543509. URL https://doi.org/10.1145/3543509. 5.3.3

[110] SAS. Sas Graphics Accelerator. https://support.sas.com/software/products/graphics-accelerator/, 2022. Accessed: 2021-09-08. 2.2.1

[111] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, January 2017. ISSN 1077-2626. doi: 10.1109/tvcg.2016.2599030. URL https://doi.org/10.1109/tvcg.2016.2599030. 1.2, 2.2.3, 5.3.2.2, 5.3.3, 5.4.3.1, 5.5.2.1, 5.8

[112] Anastasia Schaadhardt, Alexis Hiniker, and Jacob O. Wobbrock. Understanding Blind Screen-Reader Users' Experiences of Digital Artboards. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, CHI '21, New York, NY, USA, May 2021. Association for Computing Machinery. Accessed: 2021-09-06. 2.2.1

[113] William Schiff and Emerson Foulke, editors. *Tactual Perception*. Cambridge University Press, Cambridge, England, March 1982. ISBN 0521240956. 2.2.1

[114] Donald Schön and John Bennett. Reflective conversation with materials, April 1996. URL https://doi.org/10.1145/229868.230044. 9.3

[115] Hanieh Shakeri, Ye Yuan, Benett Axtell, Denise Y. Geiskkovitch, and Carman Neustaedter. Designing smart home technology for passive co-presence over distance. In *Designing Interactive Systems Conference*, DIS '24, pages 3389–3406. ACM, July 2024. doi: 10.1145/3643834.3661508. URL http://dx.doi.org/10.1145/3643834.3661508. 10.3

[116] Ather Sharif and Babak Forouraghi. evographs — a jquery plugin to create web accessible graphs. In *2018 15th IEEE Annual Consumer Communications & Networking Conference (CCNC)*, page 1–4. IEEE, January 2018. doi: 10.1109/ccnc.2018.8319239. URL https://doi.org/10.1109/ccnc.2018.8319239. 5.3.1.1

[117] Ather Sharif, Sanjana Shivani Chintalapati, Jacob O. Wobbrock, and Katharina Reinecke. Understanding screen-reader users' experiences with online data visualizations. In *Proceedings of the 23rd International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '21, page 1–16. ACM, October 2021. doi: 10.1145/3441852.3471202. URL https://doi.org/10.1145/3441852.3471202. 2.2.1, 5.2, 5.3.1.2

[118] Ather Sharif, Olivia H. Wang, Alida T. Muongchan, Katharina Reinecke, and Jacob O. Wobbrock. Voxlens: Making online data visualizations accessible with an interactive javascript plug-in. In *CHI Conference on Human Factors in Computing Systems*, CHI '22, page 1–19. ACM, April 2022. doi: 10.1145/3491102.3517431. URL https://doi.org/10.1145/3491102.3517431. 2.2.1, 5.3.1.1

[119] Ather Sharif, Joo Gyeong Kim, Jessie Zijia Xu, and Jacob O. Wobbrock. Understanding and reducing the challenges faced by creators of accessible online data visualizations. In *The 26th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '24, page 1–20. ACM, October 2024. doi: 10.1145/3663548.3675625. URL https://doi.org/10.1145/3663548.3675625. 2.2.3, 3

[120] Ashley Shew. *Against Technoableism: Rethinking Who Needs Improvement*. W. W. Norton, New York, NY, 2023. ISBN 9781324036661. ISBN: 978-1-324-03666-1. 2.3

[121] Ben Shneiderman. The eyes have it: A task by data type taxonomy for information visualizations. In *The Craft of Information Visualization*, page 364–371. Elsevier, 2003. ISBN 9781558609150. doi: 10.1016/b978-155860915-0/50046-9. URL https://doi.org/10.1016/b978-155860915-0/50046-9. 5.5.2

[122] Alexandru-Ionut Siean and Radu-Daniel Vatavu. Wearable interactions for users with motor impairments: Systematic review, inventory, and research implications. In *Proceedings of the 23rd International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '21, page 1–15. ACM, October 2021. doi: 10.1145/3441852.3471212. URL https://doi.org/10.1145/3441852.3471212. 5.3.3

[123] Volker Sorge. Polyfilling accessible chemistry diagrams. In *Lecture Notes in Computer Science*, page 43–50. Springer International Publishing, 2016. ISBN 9783319412641. doi: 10.1007/978-3-319-41264-1_6. URL https://doi.org/10.1007/978-3-319-41264-1_6. 2.2.1, 5.2, 5.3.1.1, 5.3.2.1

[124] Katta Spiel, Kathrin Gerling, Cynthia L. Bennett, Emeline Brulé, Rua M. Williams, Jennifer Rode, and Jennifer Mankoff. Nothing about us without us: Investigating the role of critical disability studies in hci. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems*, CHI '20, page 1–8. ACM, April 2020. doi: 10.1145/3334480.3375150. URL https://doi.org/10.1145/3334480.3375150. 1.1, 2.3, 10.1

[125] Clay Spinuzzi. The methodology of participatory design. *Technical communication*, 52 (2):163–174, 2005. 2.1.1

[126] Sebastian Paul Suggate. Beyond self-report: Measuring visual, auditory, and tactile mental imagery using a mental comparison task. *Behavior Research Methods*, 56(8):8658–8676, September 2024. ISSN 1554-3528. doi: 10.3758/s13428-024-02496-z. URL https://doi.org/10.3758/s13428-024-02496-z. 1.1

[127] Cella M Sum, Rahaf Alharbi, Franchesca Spektor, Cynthia L Bennett, Christina N Harrington, Katta Spiel, and Rua Mae Williams. Dreaming disability justice in hci. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, CHI '22, page 1–5. ACM, April 2022. doi: 10.1145/3491101.3503731. URL http:

//dx.doi.org/10.1145/3491101.3503731. 2.3

[128] Cella M. Sum, Franchesca Spektor, Rahaf Alharbi, Leya Breanna Baltaxe-Admony, Erika Devine, Hazel Anneke Dixon, Jared Duval, Tessa Eagle, Frank Elavsky, Kim Fernandes, Leandro S. Guedes, Serena Hillman, Vaishnav Kameswaran, Lynn Kirabo, Tamanna Motahar, Kathryn E. Ringland, Anastasia Schaadhardt, Laura Scheepmaker, and Alicia Williamson. Challenging ableism: A critical turn toward disability justice in hci. *XRDS: Crossroads, The ACM Magazine for Students*, 30(4):50–55, June 2024. ISSN 1528-4980. doi: 10.1145/3665602. URL http://dx.doi.org/10.1145/3665602. 2.3

[129] Sarit Felicia Anais Szpiro, Shafeka Hashash, Yuhang Zhao, and Shiri Azenkot. How people with low vision access computing devices: Understanding challenges and opportunities. In *Proceedings of the 18th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS '16, page 171–180, New York, NY, USA, October 2016. ACM. ISBN 9781450341240. doi: 10.1145/2982142.2982168. URL https://doi.org/10.1145/2982142.2982168. 2.2.3

[130] Pierre Tchounikine. Designing for appropriation: A theoretical account. *Human–Computer Interaction*, 32(4):155–195, July 2016. ISSN 1532-7051. doi: 10.1080/07370024.2016.1203263. URL https://doi.org/10.1080/07370024.2016.1203263. 2.1.2

[131] Yuanyang (YY) Teng and Darren Gergle. Access is not enough: Toward developmental flourishing. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems*, CHI EA '26, page 1–6. ACM, April 2026. doi: 10.1145/3772363.3798552. URL http://dx.doi.org/10.1145/3772363.3798552. 1.1

[132] John R Thompson, Jesse J Martinez, Alper Sarikaya, Edward Cutrell, and Bongshin Lee. Chart reader: Accessible visualization experiences designed with screen reader users. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, page 1–18. ACM, April 2023. doi: 10.1145/3544548.3581186. URL https://doi.org/10.1145/3544548.3581186. 2.2.1, 2.4.1, 5.2, 5.3.1.1, 5.4.1.1, 5.5.3

[133] Alexandra To, Angela D. R. Smith, Dilruba Showkat, Adinawa Adjagbodjou, and Christina Harrington. Flourishing in the everyday: Moving beyond damage-centered design in hci for bipoc communities. In *Proceedings of the 2023 ACM Designing Interactive Systems Conference*, DIS '23, pages 917–933. ACM, July 2023. doi: 10.1145/3563657.3596057. URL https://doi.org/10.1145/3563657.3596057. 2.1.1

[134] Jacob VanderPlas, Brian Granger, Jeffrey Heer, Dominik Moritz, Kanit Wongsuphasawat, Arvind Satyanarayan, Eitan Lees, Ilia Timofeev, Ben Welsh, and Scott Sievert. Altair: Interactive statistical visualizations for python. *Journal of Open Source Software*, 3(32): 1057, December 2018. ISSN 2475-9066. doi: 10.21105/joss.01057. URL http://dx.doi.org/10.21105/joss.01057. 5.8

[135] Visa. Visa Chart Components. https://github.com/visa/visa-chart-components, 2022. Accessed: 2022-12-01. 2.2.1, 5.1.1, 5.3.2.1

[136] W3C. Web content accessibility guidelines (wcag) 2.1.

https://www.w3.org/TR/WCAG21/, 2018. Accessed: 2021-09-03. 2.2.2, 3

[137] WAI. Understanding success criterion 2.4.7: focus-visible. WCAG standard, W3C, 2016. Accessed: 2022-12-11. 5.4.3.2

[138] WAI. Understanding success criterion 4.1.2: name, role, value. WCAG standard, W3C, 2016. Accessed: 2022-03-04. 2.2.2, 5.3.1.3

[139] WAI. Accessible rich internet applications (WAI-ARIA 1.1). Technical report, W3C, 2017. Accessed: 2022-12-11. 5.1.1, 3

[140] WAI. Understanding success criterion 2.1.1: keyboard. WCAG standard, W3C, 2017. URL https://www.w3.org/WAI/WCAG21/Understanding/keyboard.html. Accessed: 2026-03-20, https://www.w3.org/WAI/WCAG21/Understanding/keyboard.html. 5.4.2.2

[141] Bruce N. Walker and Michael A. Nees. Theory of sonification. In Thomas Hermann, Andy Hunt, and John G. Neuhoff, editors, *The Sonification Handbook*, chapter 2, pages 9–39. Logos Verlag, Berlin, Germany, 2011. 2.2.1

[142] Chris Weaver. Multidimensional visual analysis using cross-filtered views. In *2008 IEEE Symposium on Visual Analytics Science and Technology*, page 163–170. IEEE, October 2008. doi: 10.1109/vast.2008.4677370. URL https://doi.org/10.1109/vast.2008.4677370. 3

[143] WebAIM. WAVE, the web accessibility evaluation tool. https://wave.webaim.org/, 2020. Accessed: 2023-01-10. 5.5.1.1

[144] WebAIM. Webaim: The WebAIM Million - An annual accessibility analysis of the top 1,000,000 home pages. https://webaim.org/projects/million/#wcag, 2021. Accessed: 2021-09-03. 10.3

[145] David Gray Widder and Richmond Wong. Thinking upstream: Ethics and policy opportunities in ai supply chains, 2023. URL https://doi.org/10.48550/arxiv.2303.07529. 1.2

[146] David Gray Widder, Meredith Whittaker, and Sarah Myers West. Why 'open' ai systems are actually closed, and why this matters. *Nature*, 635(8040):827–833, November 2024. ISSN 1476-4687. doi: 10.1038/s41586-024-08141-1. URL https://doi.org/10.1038/s41586-024-08141-1. 1.2

[147] Rua M. Williams, Kathryn Ringland, Amelia Gibson, Mahender Mandala, Arne Maibaum, and Tiago Guerreiro. Articulations toward a crip hci. *Interactions*, 28(3):28–37, April 2021. ISSN 1558-3449. doi: 10.1145/3458453. URL http://dx.doi.org/10.1145/3458453. 2.3

[148] B. L. Wimer, L. South, K. Wu, D. A. Szafir, M. A. Borkin, and R. A. Metoyer. Beyond vision impairments: Redefining the scope of accessible data representations. *IEEE Transactions on Visualization and Computer Graphics*, 30(12):7619–7636, December 2024. 2.2.3

[149] Langdon Winner. Do artifacts have politics? In *Computer Ethics*, page 177–192. Routledge, May 2017. ISBN 9781315259697. doi: 10.4324/9781315259697-21. URL

https://doi.org/10.4324/9781315259697-21. 1.2

[150] Jacob O. Wobbrock, Shaun K. Kane, Krzysztof Z. Gajos, Susumu Harada, and Jon Froehlich. Ability-based design: Concept, principles and examples. *ACM Transactions on Accessible Computing*, 3(3):1–27, April 2011. ISSN 1936-7236. doi: 10.1145/1952383.1952384. URL https://doi.org/10.1145/1952383.1952384. 1.2, 2.1.2, 2.4.3, 5.6

[151] Mintesnot Woldeamanuel and Andrew Kent. Measuring walk access to transit in terms of sidewalk availability, quality, and connectivity. *Journal of Urban Planning and Development*, 142(2), June 2016. ISSN 1943-5444. doi: 10.1061/(asce)up.1943-5444.0000296. URL http://dx.doi.org/10.1061/(ASCE)UP.1943-5444.0000296. 10.3

[152] R. Wood et al. "creatures of habit": influential factors to the adoption of computer personalization and accessibility settings. *Universal Access in the Information Society*, 23(2): 927–953, March 2023. 2.4.3, 3

[153] Katherine Ye, Wode Ni, Max Krieger, Dor Ma'ayan, Jenna Wise, Jonathan Aldrich, Joshua Sunshine, and Keenan Crane. Penrose: from mathematical notation to beautiful diagrams. *ACM Transactions on Graphics*, 39(4), August 2020. ISSN 1557-7368. doi: 10.1145/3386569.3392375. URL https://doi.org/10.1145/3386569.3392375. 5.5.3.1

[154] Rebecca Yeo and Karen Moore. Including disabled people in poverty reduction work: "nothing about us, without us". *World Development*, 31(3):571–590, March 2003. ISSN 0305-750X. doi: 10.1016/s0305-750x(02)00218-8. URL https://doi.org/10.1016/s0305-750x(02)00218-8. 1.1

[155] Haixia Zhao, Catherine Plaisant, Ben Shneiderman, and Jonathan Lazar. Data sonification for users with visual impairment: A case study with georeferenced data. *ACM Transactions on Computer-Human Interaction*, 15(1):1–28, May 2008. ISSN 1557-7325. doi: 10.1145/1352782.1352786. URL https://doi.org/10.1145/1352782.1352786. 2.2.1

[156] Jonathan Zong. Using real names of disabled participant-contributors to practice citational justice in accessibility. In *2025 IEEE Workshop on Accessible Data Visualization (AccessViz)*, page 30–33. IEEE, November 2025. doi: 10.1109/accessviz68666.2025.00011. URL https://doi.org/10.1109/accessviz68666.2025.00011. 2.3

[157] Jonathan Zong, Crystal Lee, Alan Lundgard, JiWoong Jang, Daniel Hajas, and Arvind Satyanarayan. Rich screen reader experiences for accessible data visualization. *Computer Graphics Forum*, 41(3):15–27, June 2022. doi: 10.1111/cgf.14519. URL https://doi.org/10.1111/cgf.14519. 2.2.1, 2.3, 2.4.1, 2.4.2, 3, 5.2, 5.3.1.1, 5.3.1.3, 5.3.2.1, 5.4.1.1, 5.4.3.3, 5.5.1.1, 5.5.2, 5.5.3, 5.6

[158] Jonathan Zong, Isabella Pedraza Pineros, Mengzhu (Katie) Chen, Daniel Hajas, and Arvind Satyanarayan. Umwelt: Accessible structured editing of multi-modal data representations. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, CHI '24, page 1–20. ACM, May 2024. doi: 10.1145/3613904.3641996. URL https://doi.org/10.1145/3613904.3641996. 2.2.1